

컴퓨터 비전

Computer Vision

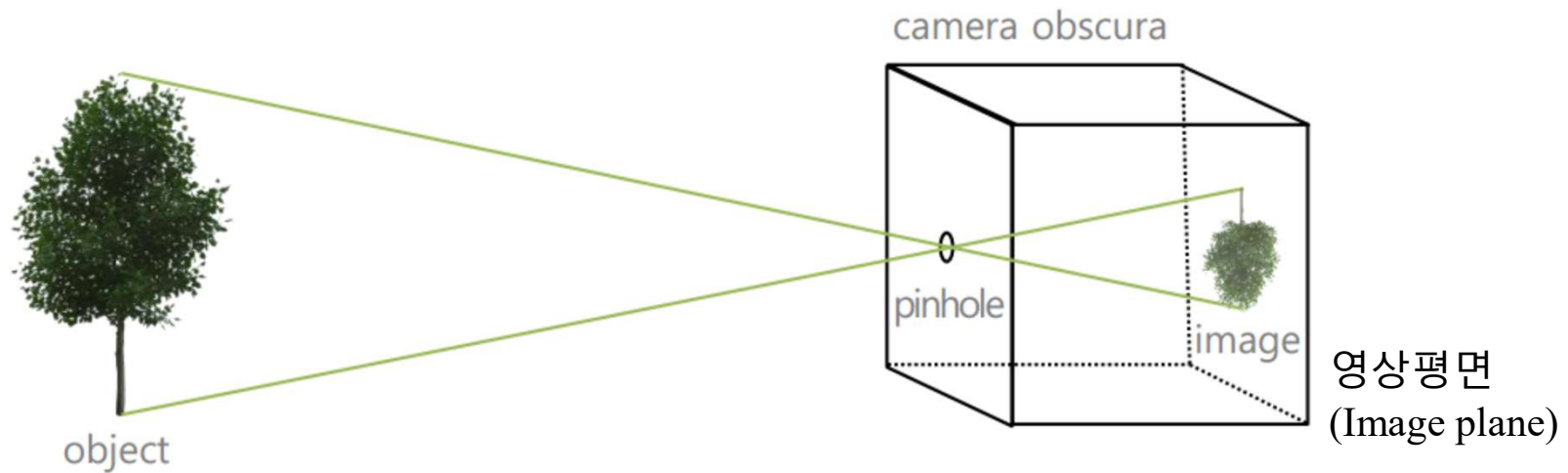
- Stereo Vision -

동아대학교 소프트웨어대학 AI학과
2026년 1학기

임 한 신

카메라 모델

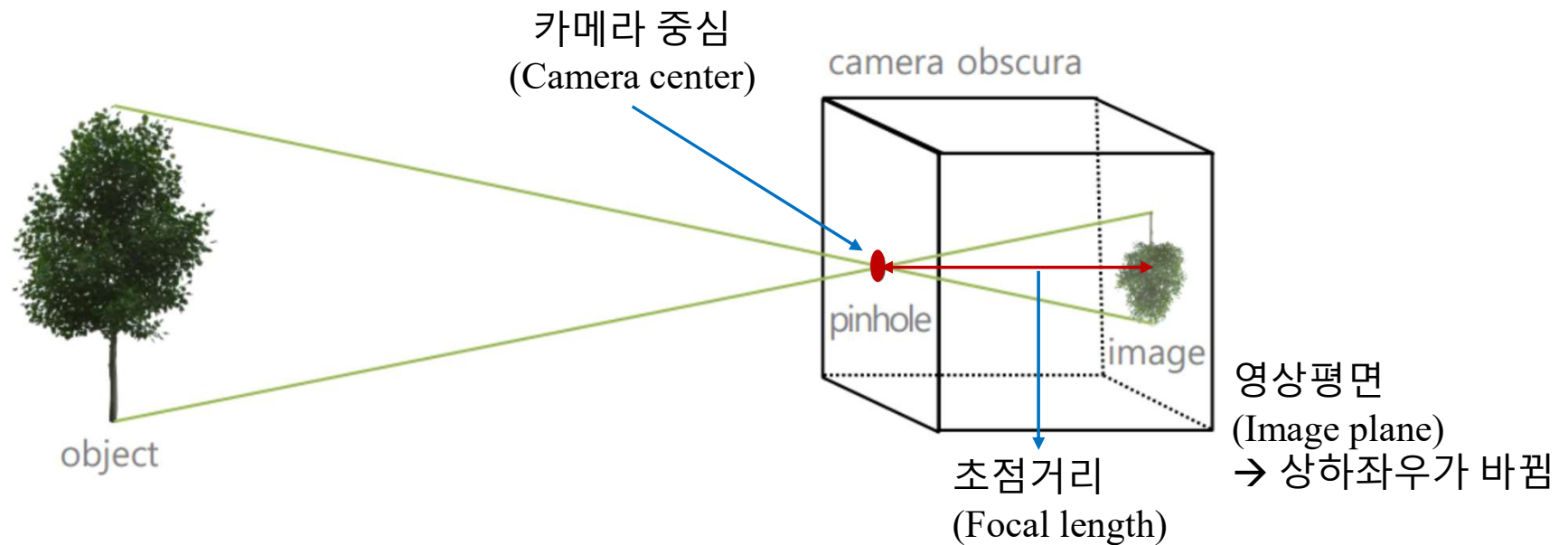
- 핀홀 카메라 모델(Pinhole camera model)



물체에서 나온 빛은 pinhole을 일직선으로 통과하여 반대편 벽(영상평면)에 상(image)을 맺힘

카메라 모델

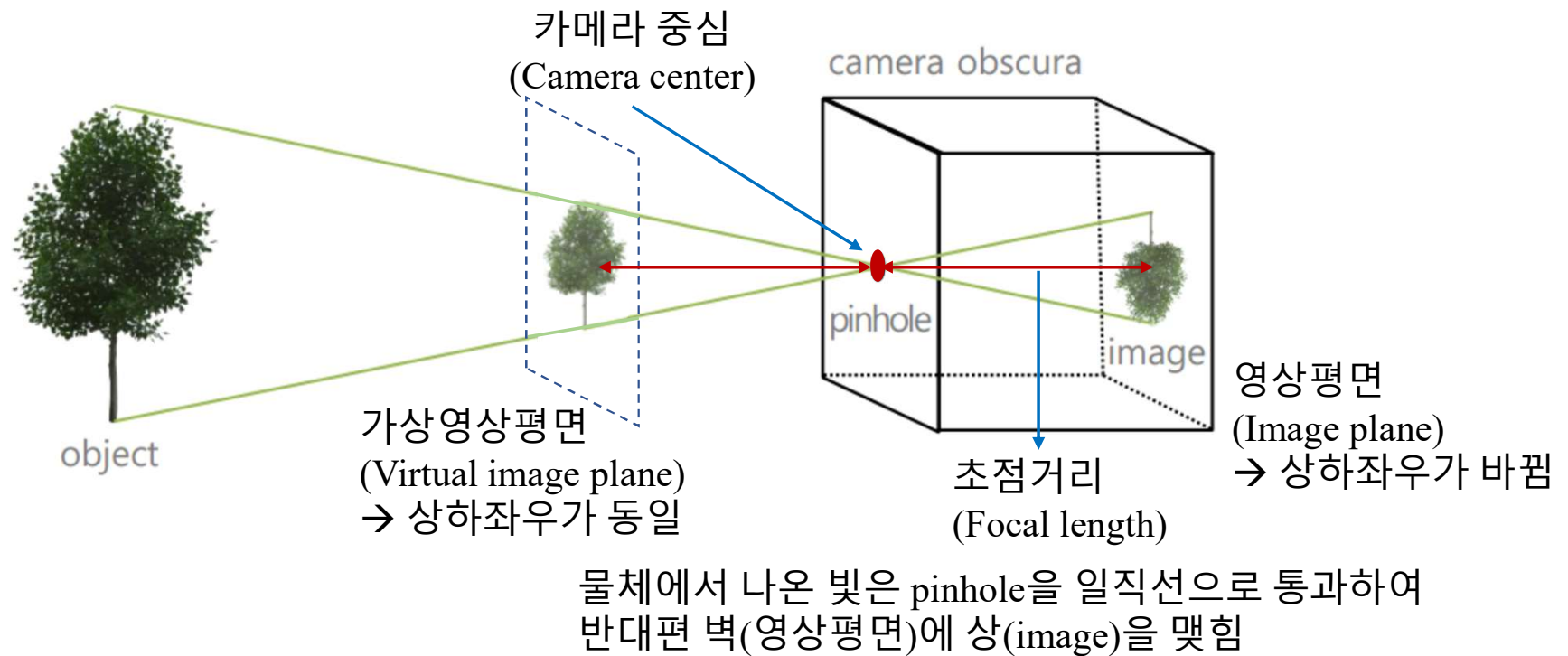
- 핀홀 카메라 모델(Pinhole camera model)



물체에서 나온 빛은 pinhole을 일직선으로 통과하여 반대편 벽(영상평면)에 상(image)을 맺힘

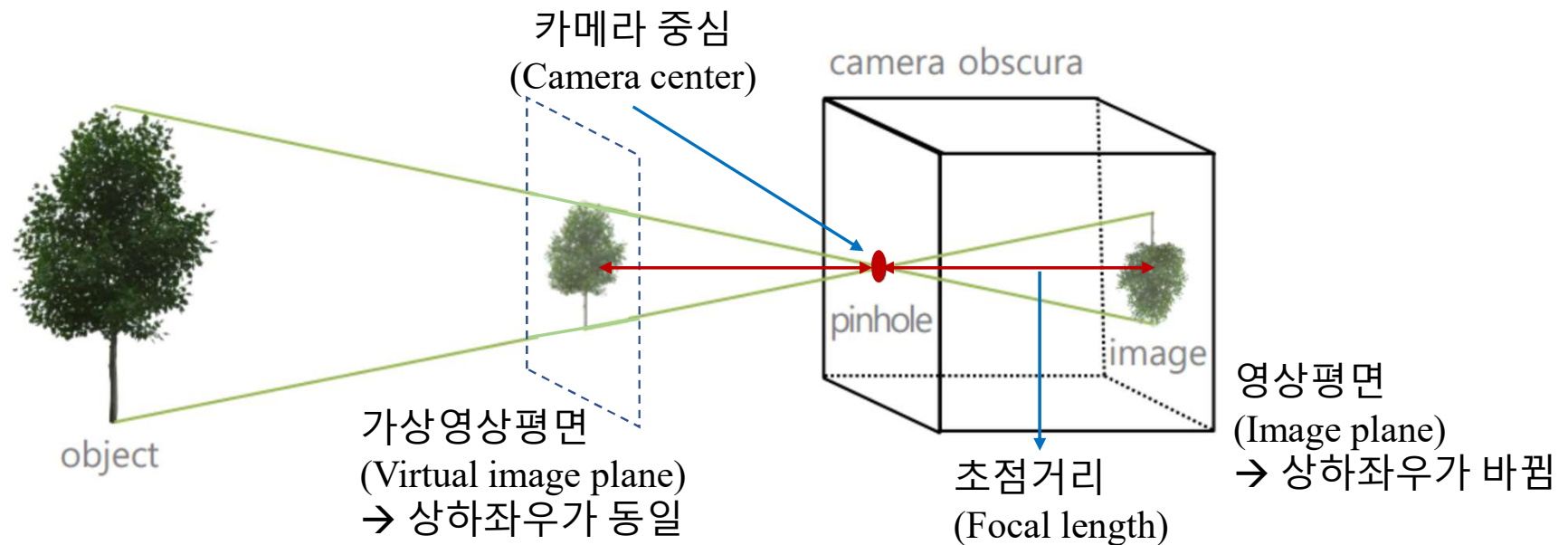
카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



카메라 모델

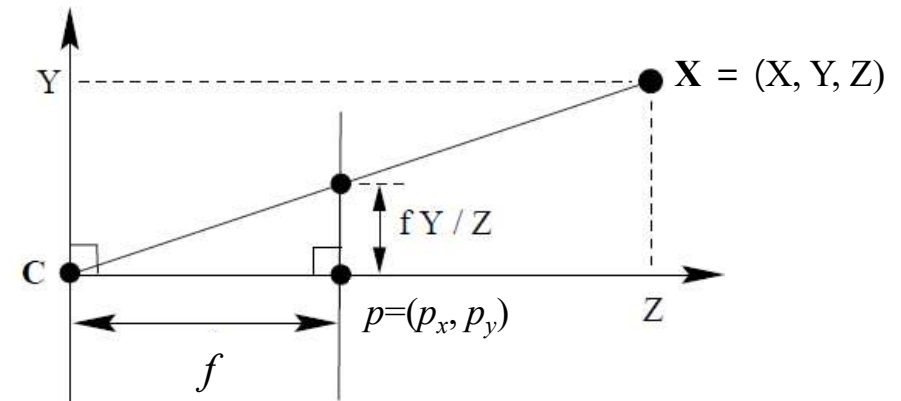
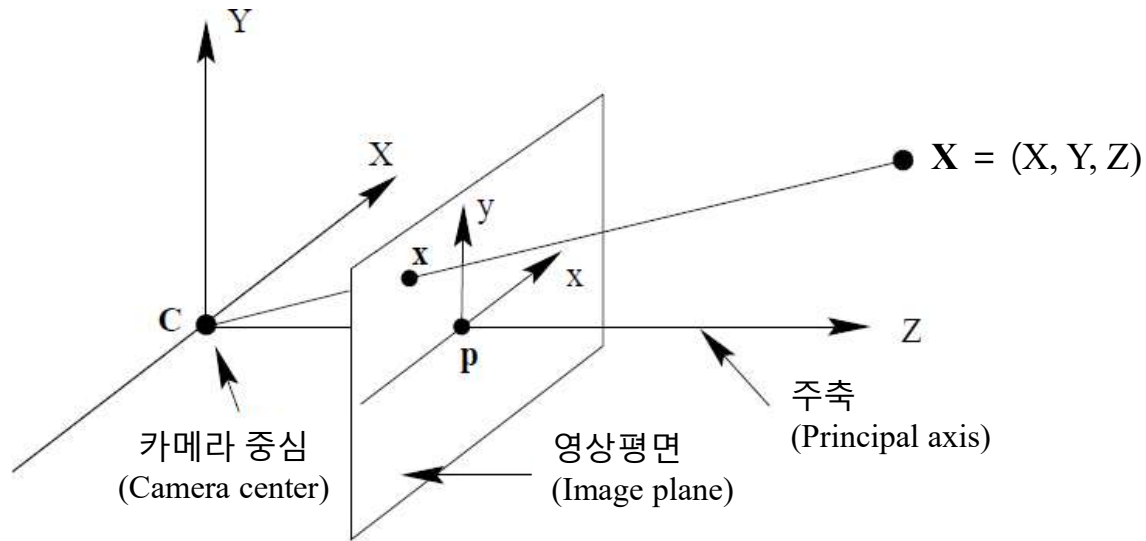
- 핀홀 카메라 모델(Pinhole camera model)



물체에서 나온 빛은 pinhole을 일직선으로 통과하여 반대편 벽(영상평면)에 상(image)을 맺힘
가상영상평면을 두면 상하좌우가 바뀌지 않으면서 영상평면과 동일한 기하학적 모델이 가능

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z-축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

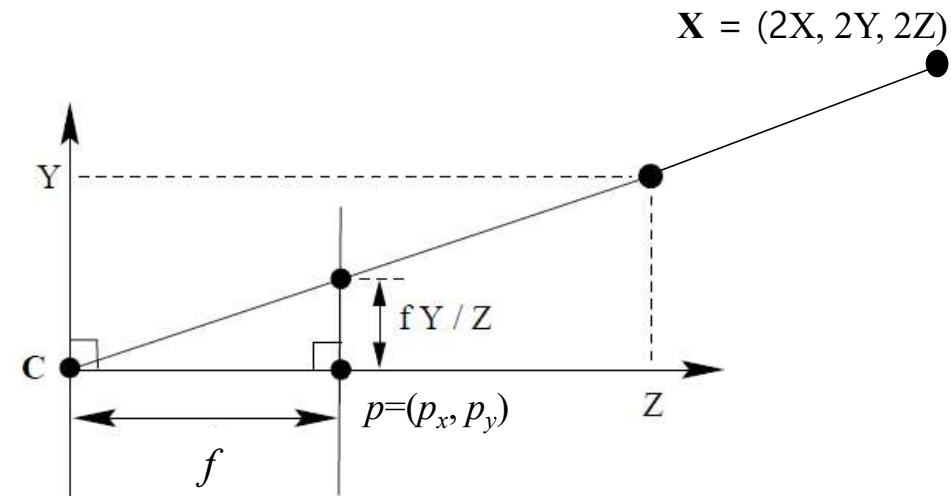
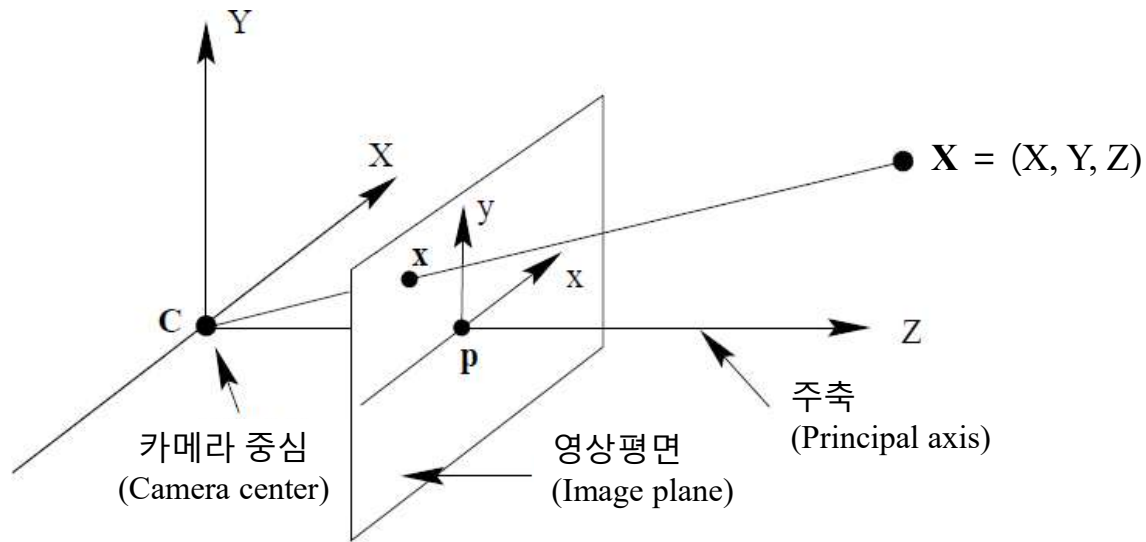
$\mathbf{x} = (x, y)$ 는 아래와 같이 계산됨

$$x = \frac{fX}{Z} + p_x$$

$$y = \frac{fY}{Z} + p_y$$

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



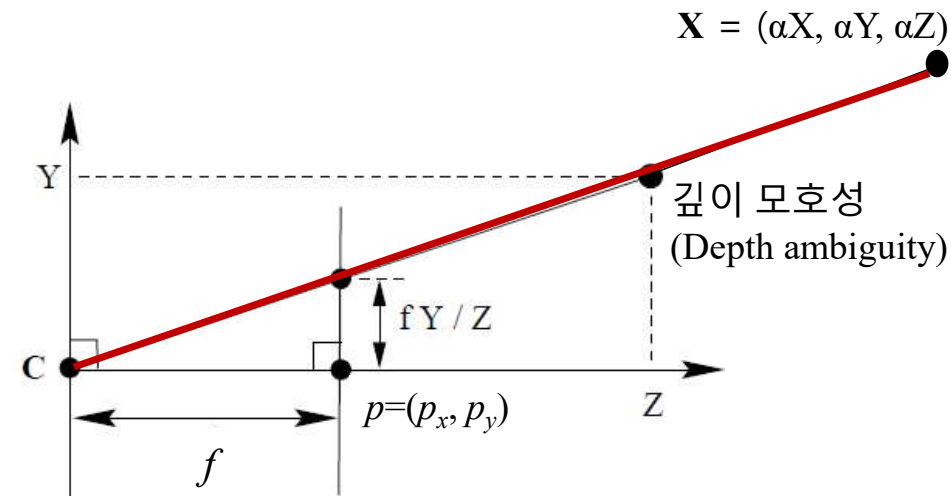
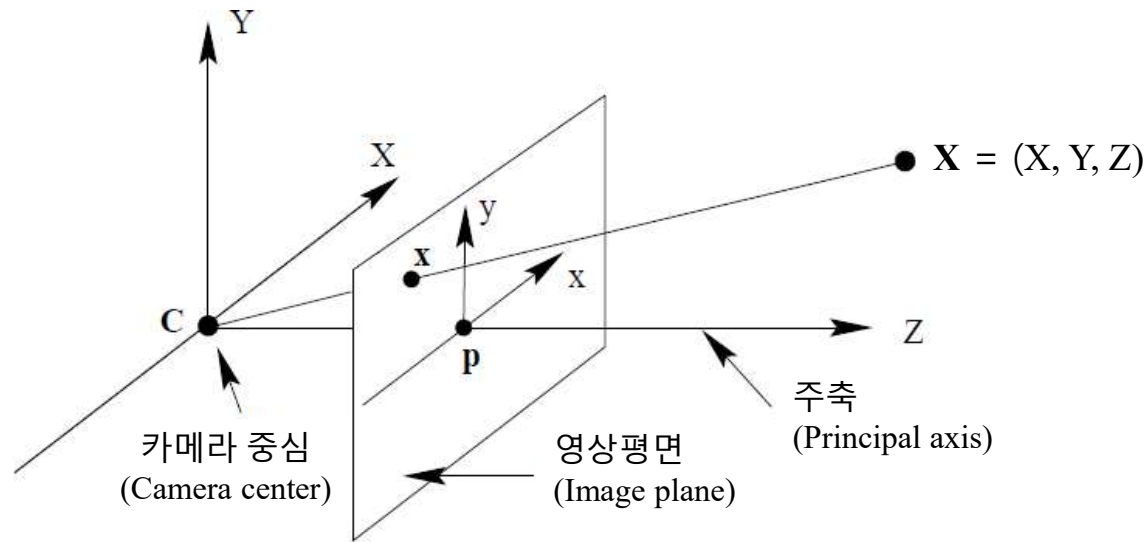
- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(Z -축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

$\mathbf{x} = (x, y)$ 는 아래와 같이 계산됨

$$x = \frac{f2X}{2Z} + p_x = \frac{fX}{Z} + p_x$$
$$y = \frac{f2Y}{2Z} + p_y = \frac{fY}{Z} + p_y$$

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z-축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

$\mathbf{x} = (x, y)$ 는 아래와 같이 계산됨

$$\begin{aligned} x &= \frac{f\alpha X}{\alpha Z} + p_x = \frac{fX}{Z} + px \\ y &= \frac{f\alpha Y}{\alpha Z} + p_y = \frac{fY}{Z} + py \end{aligned}$$

카메라 모델

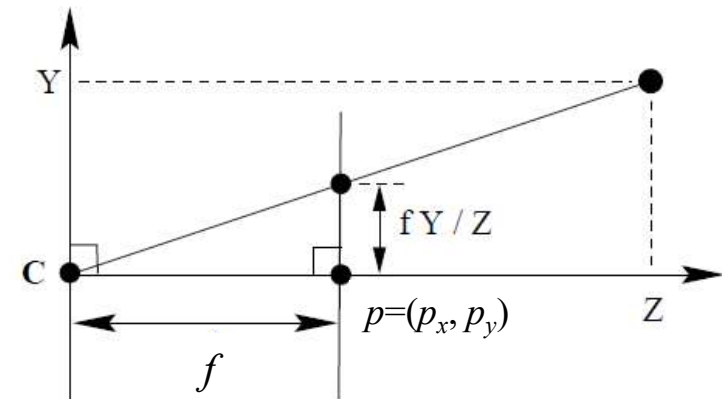
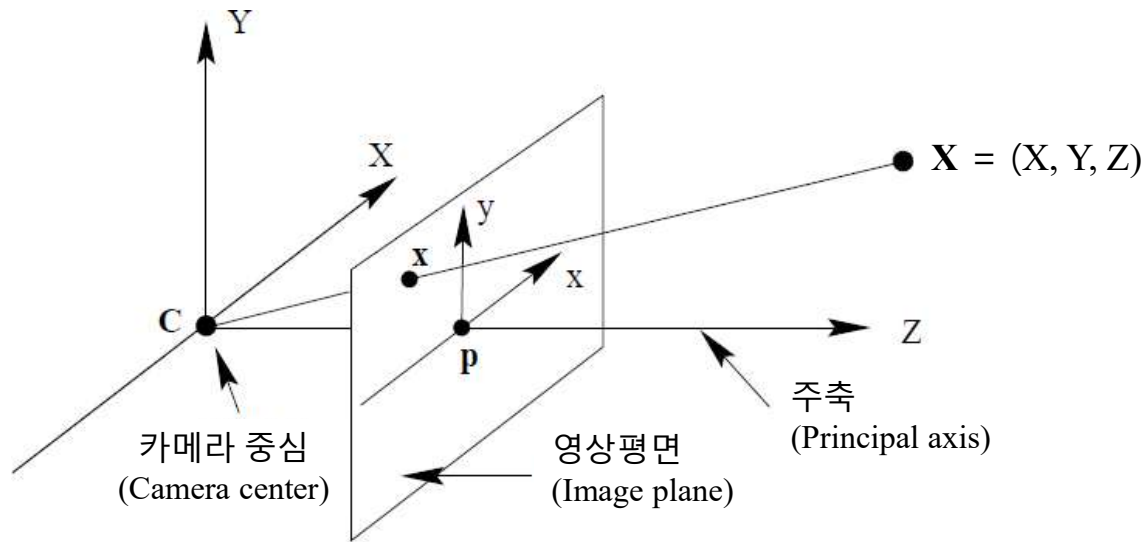
- 핀홀 카메라 모델(Pinhole camera model)



깊이 모호성(Depth ambiguity)

카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



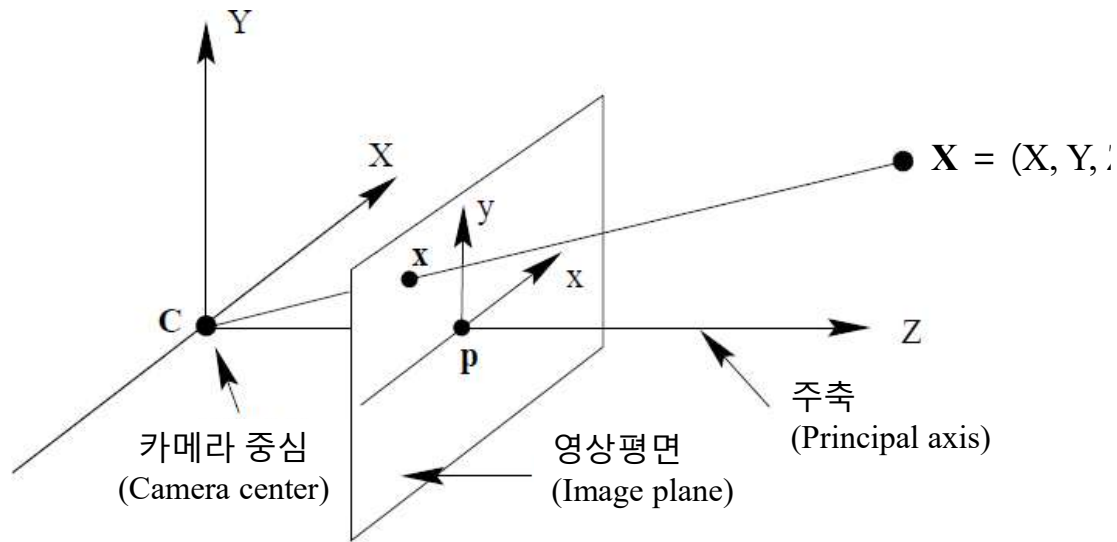
- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(Z -축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

카메라 내부파라미터

- 초점거리(Focal length(단위 : pixels))
- 주점(Principal point(단위 : pixels))

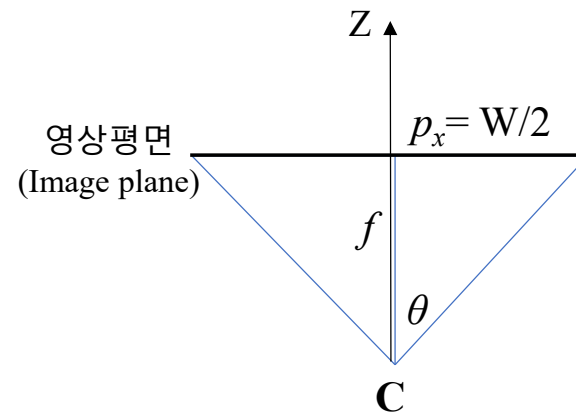
카메라 모델

- 핀홀 카메라 모델(Pinhole camera model)



- 주축(Principal axis) : 카메라 중심에서 영상 평면에 수직으로 내린 선(z-축으로 설정됨)
- 주점(Principal point) : 주축이 영상 평면과 만나는 교점. 영상의 정중앙 또는 근처에 위치

카메라 화각 계산



$$\text{수평 화각} = 2\theta = 2 \times \arctan(W / 2f)$$

예) $W : 1920$, f : 약 1330 라고 하면

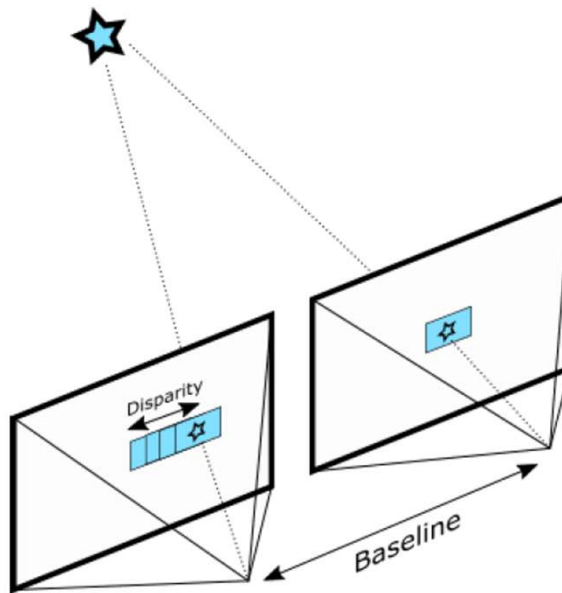
수평 화각은 약 71.6도 정도 나옴

스테레오 비전(Stereo Vision)

- 두 개의 카메라(스테레오 카메라)를 이용해 마치 사람의 두 눈처럼 사물의 입체감과 거리(깊이, Depth)를 추정하는 기술
- Disparity(양안 시차) : 두 눈(또는 두 카메라) 사이의 시점 차이로 인해 발생하는 영상 간의 위치 차이
- 참고 : 인간의 눈은 약 6.5cm 떨어져 있음



스테레오 카메라



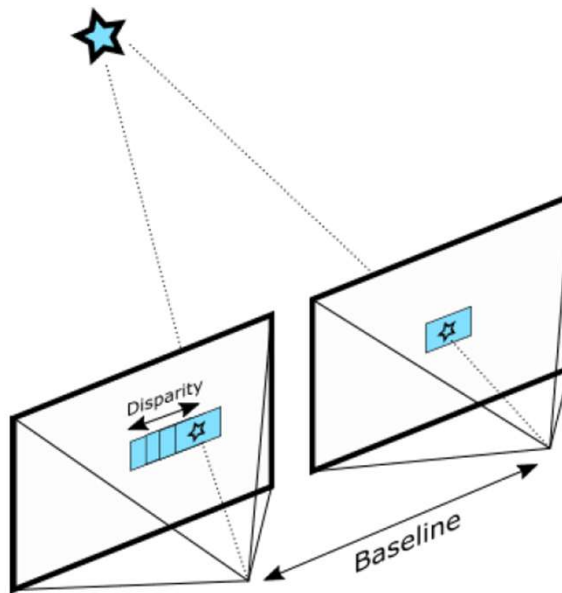
Left image

스테레오 비전(Stereo Vision)

- 두 개의 카메라(스테레오 카메라)를 이용해 마치 사람의 두 눈처럼 사물의 입체감과 거리(깊이, Depth)를 추정하는 기술
- Disparity(양안 시차) : 두 눈(또는 두 카메라) 사이의 시점 차이로 인해 발생하는 영상 간의 위치 차이
- 참고 : 인간의 눈은 약 6.5cm 떨어져 있음



스테레오 카메라



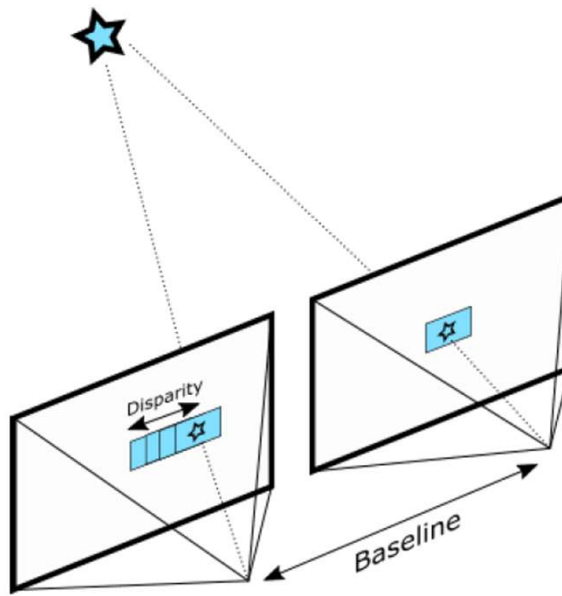
Right image

스테레오 비전(Stereo Vision)

- 두 개의 카메라(스테레오 카메라)를 이용해 마치 사람의 두 눈처럼 사물의 입체감과 거리(깊이, Depth)를 추정하는 기술
- Disparity(양안 시차) : 두 눈(또는 두 카메라) 사이의 시점 차이로 인해 발생하는 영상 간의 위치 차이
- 참고 : 인간의 눈은 약 6.5cm 떨어져 있음



스테레오 카메라



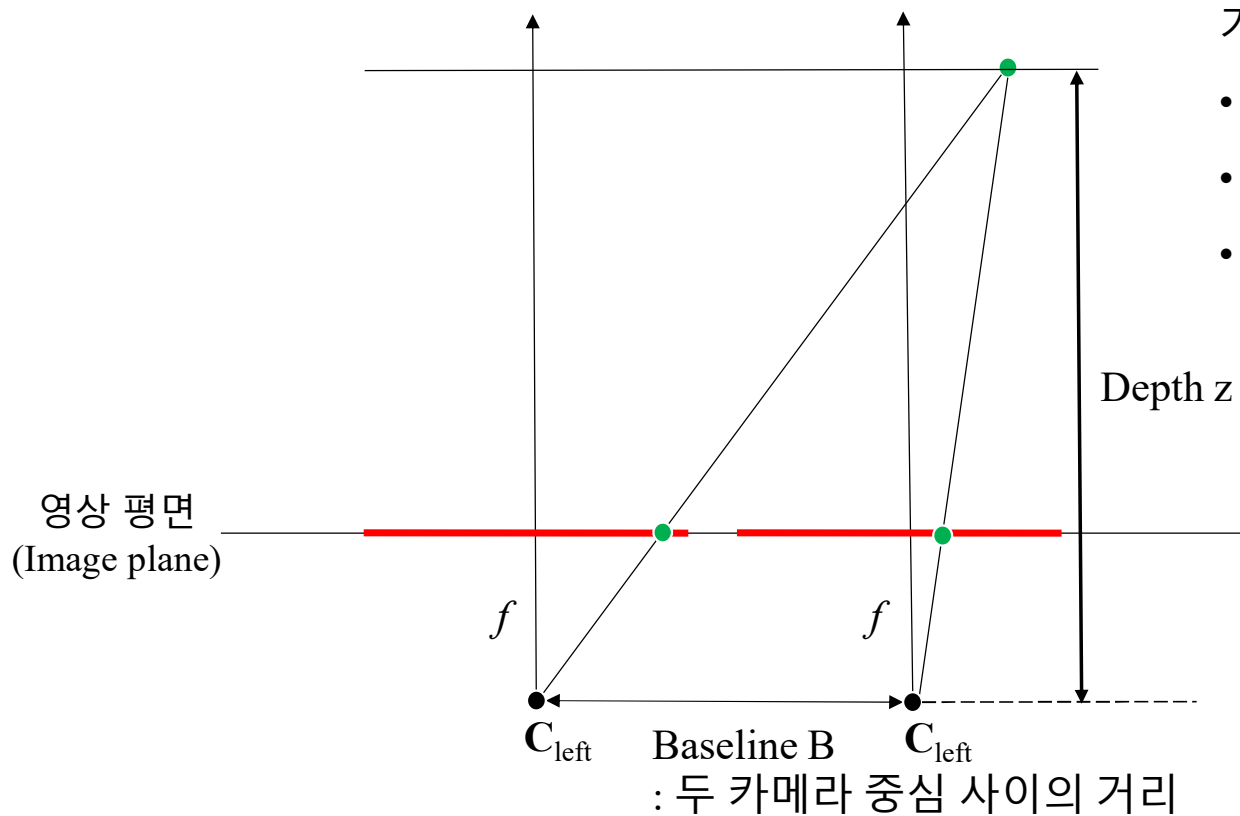
Left image



Right image

카메라 모델

- Disparity(양안시차)와 depth

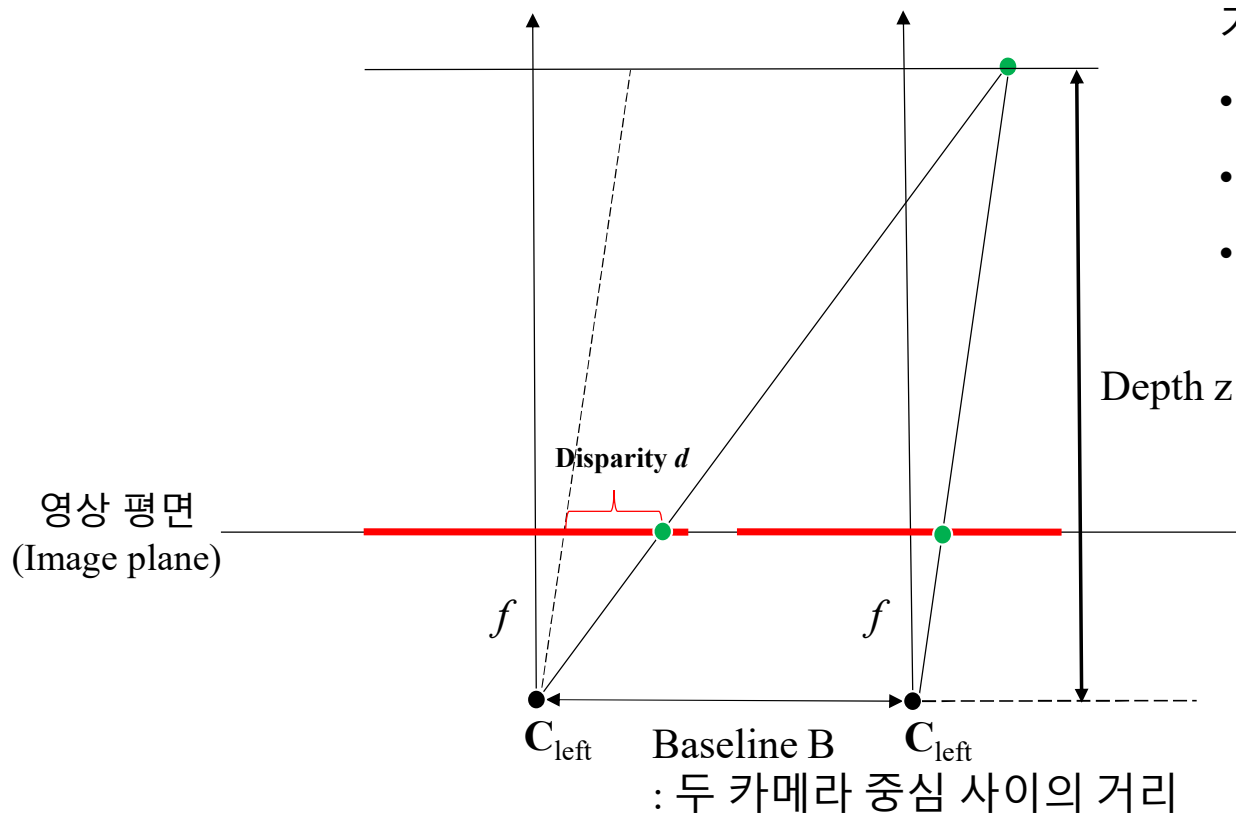


가정

- Baseline $B \neq 0$
- $f_{\text{left}} = f_{\text{right}} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

카메라 모델

- Disparity(양안시차)와 depth

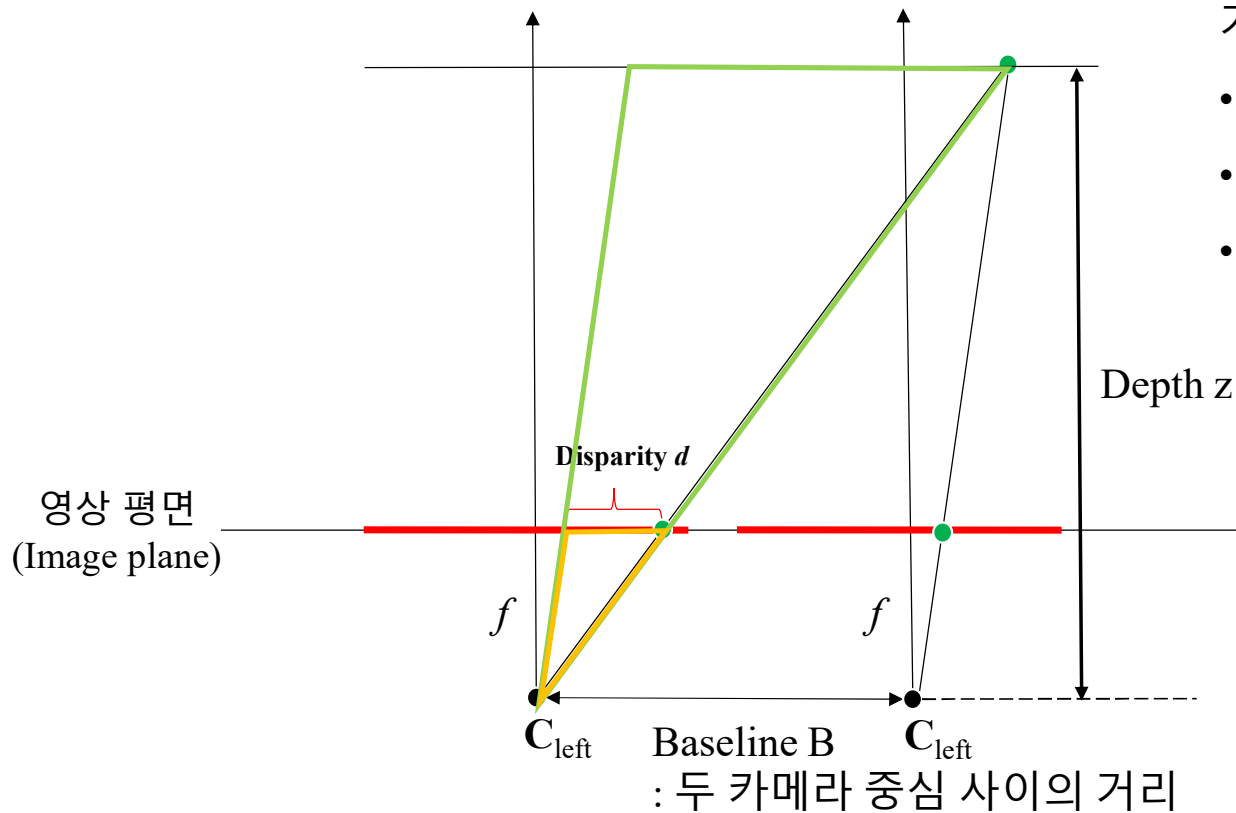


가정

- Baseline $B \neq 0$
- $f_{left} = f_{right} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

카메라 모델

- Disparity(양안시차)와 depth



가정

- Baseline $B \neq 0$
- $f_{left} = f_{right} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

$$B : d = z : f$$

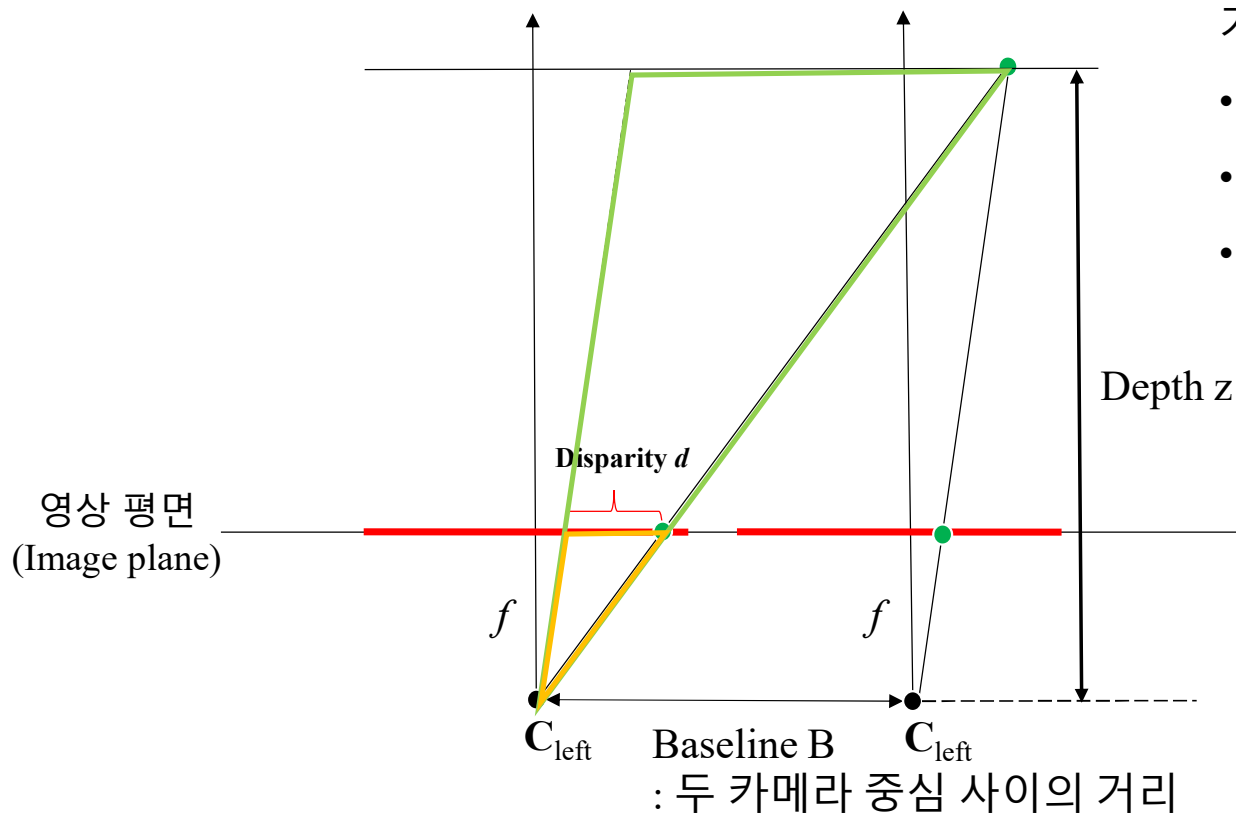
$$z = \frac{fB}{d}$$

$$z \propto \frac{1}{d}$$

$$z \propto fB$$

카메라 모델

- Disparity(양안시차)와 depth



가정

- Baseline $B \neq 0$
- $f_{left} = f_{right} = f$
- 두 영상의 영상 평면은 동일한 평면 상에 있음

$$B : d = z : f$$

$$z = \frac{fB}{d}$$

예) $f = 1000$, baseline $B = 65\text{mm}$, $d = 50$

$\rightarrow z = 1300 \text{ mm}$

카메라 모델

- 스테레오 영상 : 일반적으로 일치점들은 영상에서 동일한 높이에 있다고 가정

Left image



Right image

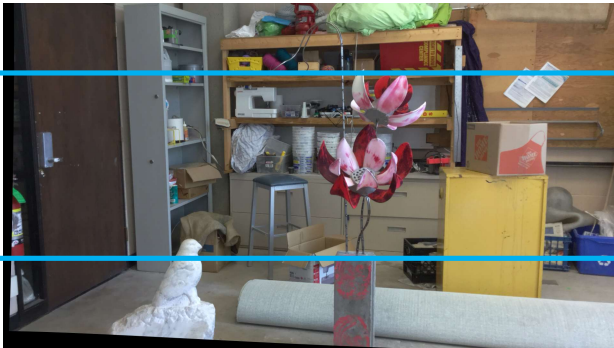


From <https://vision.middlebury.edu/stereo/data/>

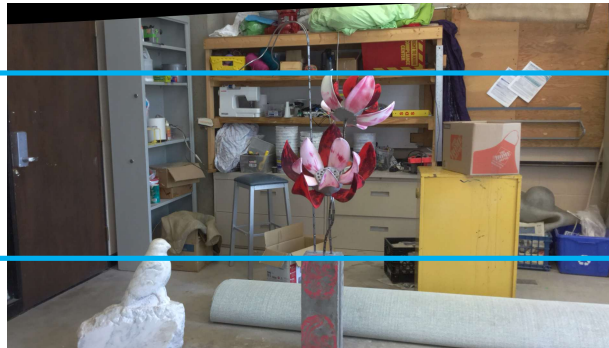
카메라 모델

- 스테레오 영상 : 일반적으로 일치점들은 영상에서 동일한 높이에 있다고 가정

Left image



Right image

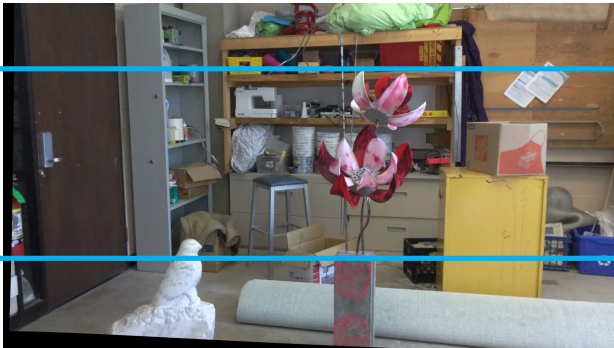


From <https://vision.middlebury.edu/stereo/data/>

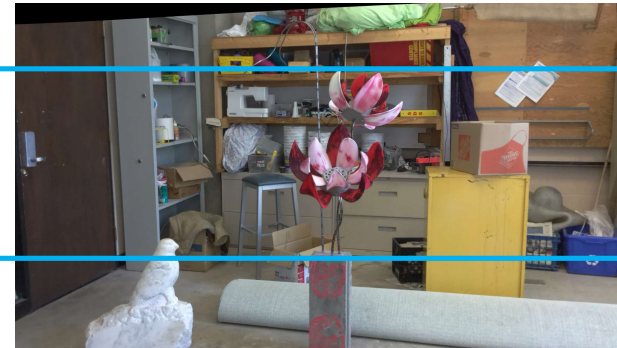
카메라 모델

- 스테레오 영상 : 일반적으로 일치점들은 영상에서 동일한 높이에 있다고 가정
- Disparity map(양안시차지도, 깊이지도, 깊이맵) : 스테레오 영상의 disparity를 픽셀 단위로 나타낸 map

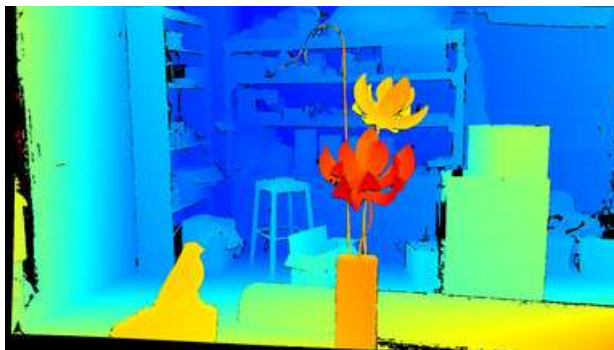
Left image



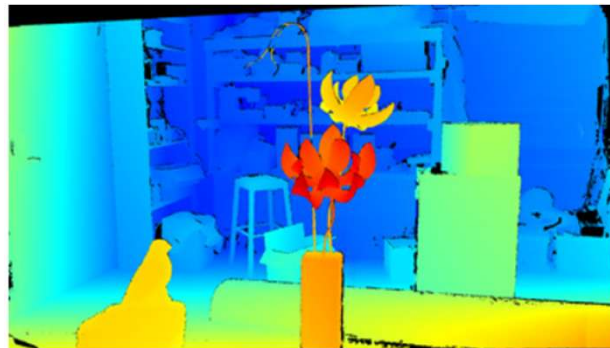
Right image



Disparity map of
left image



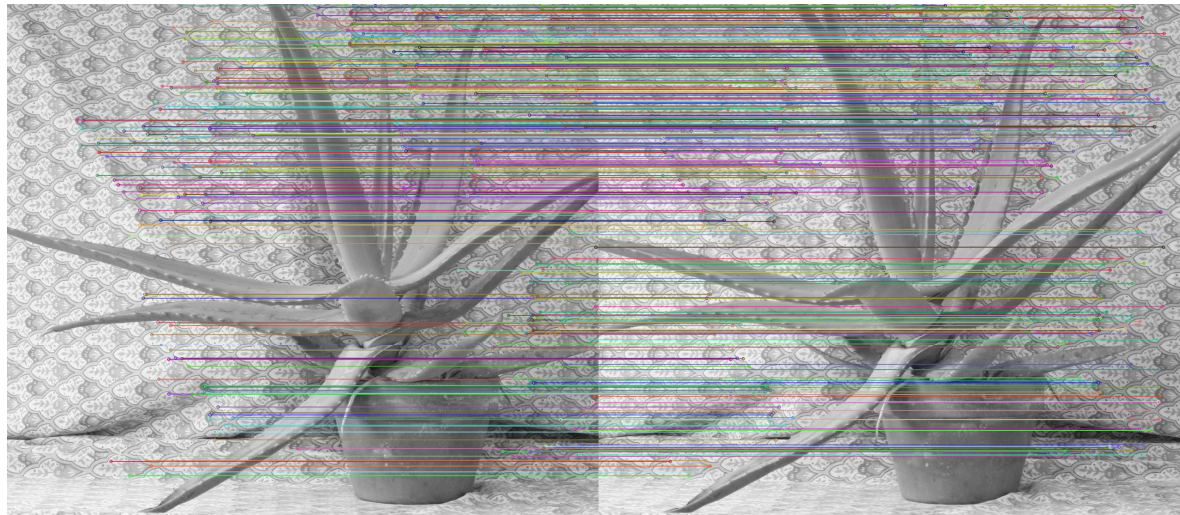
Disparity map of
right image



From <https://vision.middlebury.edu/stereo/data/>

Disparity Estimation(양안시차 추정)

- Disparity estimation(양안시차 추정) → depth estimation(깊이 추정)
- SIFT, ORB 등 특징 추출 기술을 이용하는 경우
→ 특징이 아니 대부분의 픽셀의 disparity 추정이 어려움



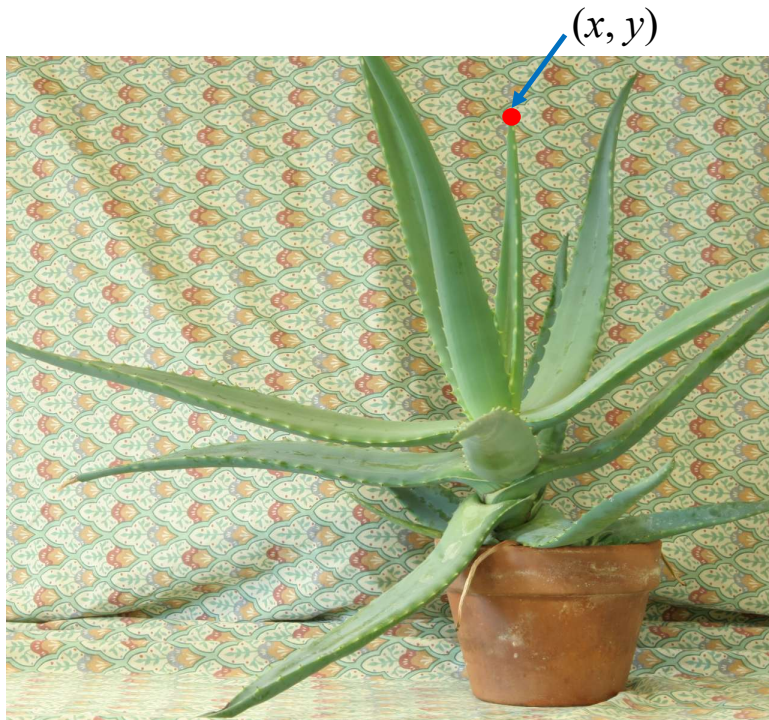
Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등
- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지를 최소화하는 방식으로 깊이지도 생성
 - GraphCut(그래프 기반 방법) 등
- Semi-global matching(세미-전역 매칭) 기반 방법
 - 블록 매칭의 속도와 전역 매칭의 정확도 사이에서 균형을 맞춘 방식
 - SGBM(Semi-Global Block Matching) 등
- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - RAFT-Stereo 등

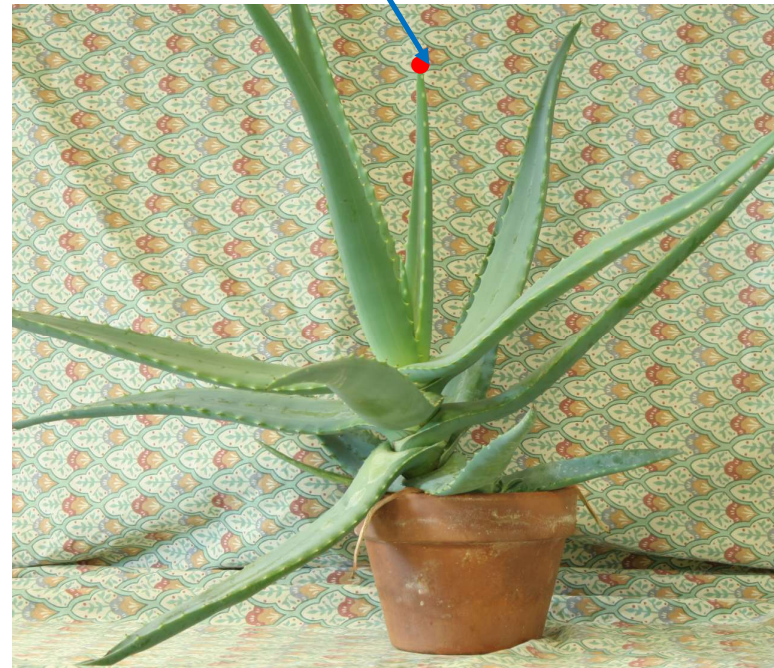
Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등

Left image



$(x-d, y)$



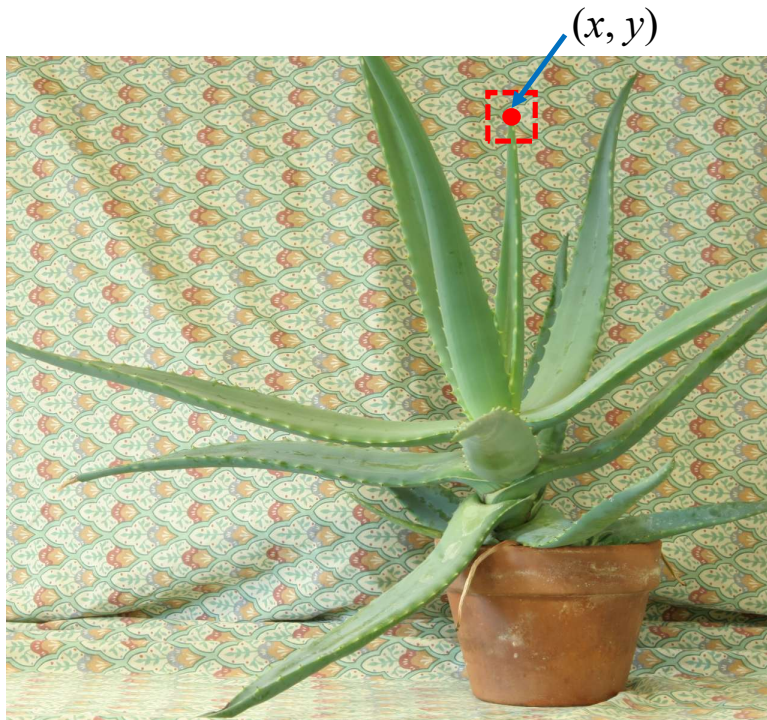
d : disparity

Right image

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등

Left image



$(x-d, y)$



 : 블록

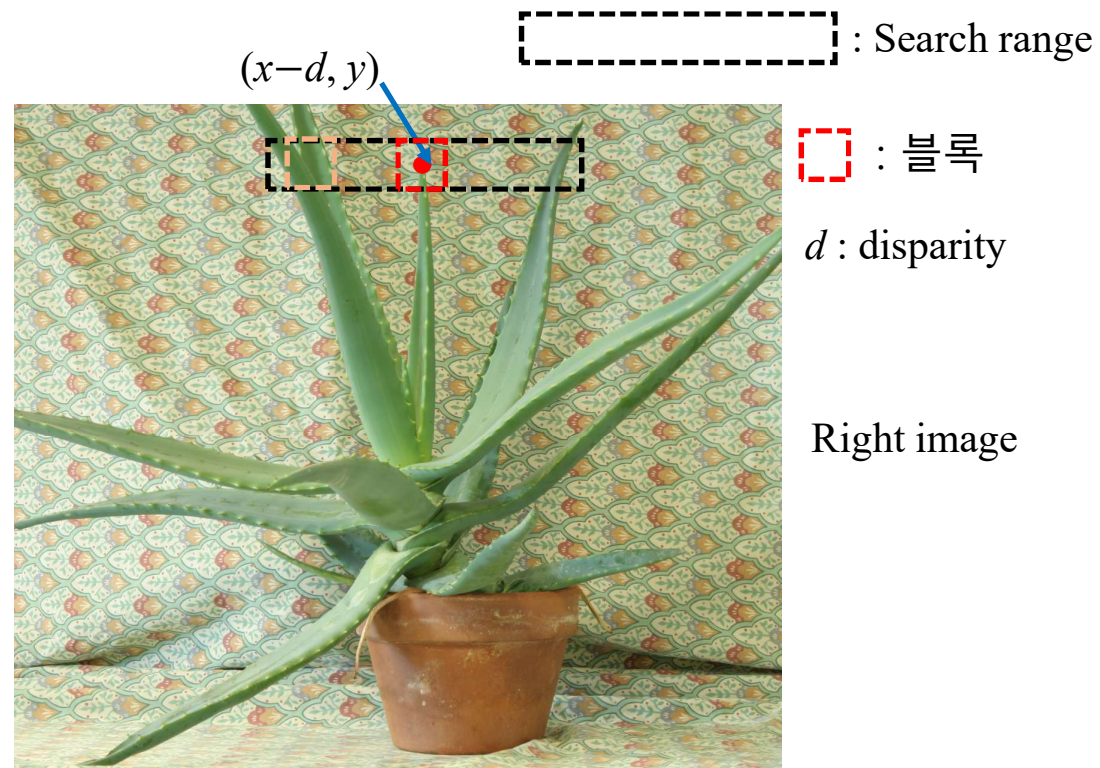
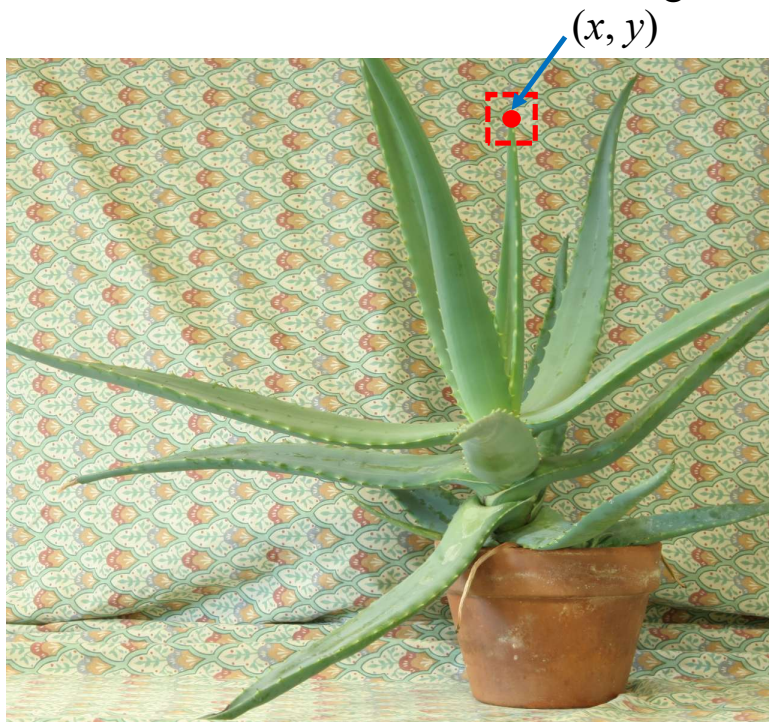
d : disparity

Right image

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD, SSD, NCC 등
 - 주요 파라미터 : 블록사이즈, search range

Left image



Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - SAD(Sum of Absolute Difference)
 - 대응하는 픽셀 값 차이의 절댓값을 모두 더함
 - 연산량이 가장 적음

$$SAD(x, y, d) = \sum_{(i,j) \in W} |I_L(x + i, y + j) - I_R(x + i - d, y + j)|$$

- SSD(Sum of Squared Difference)
 - 픽셀 값 차이를 제곱하여 더함
 - 차이가 큰 픽셀(노이즈)에 대해 더 큰 페널티를 부여

$$SSD(x, y, d) = \sum_{(i,j) \in W} (I_L(x + i, y + j) - I_R(x + i - d, y + j))^2$$

3x3 인 블록(W)의 (i,j)예시

-1, -1	0, -1	1, -1
-1, 0	0, 0	1, 0
-1, 1	0, 1	1, 1

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - NCC(Normalized Cross Correlation)
 - 분자에는 두 블록 사이의 공분산, 분모에는 각 블록의 표준편차의 곱이 들어감
 - $-1 \leq NCC \leq 1$ 이며, 1에 가까울수록 두 패치가 완벽하게 일치함을 뜻함
 - 두 블록의 상관계수(correlation coefficient)를 구하는 것과 같음

3x3 인 블록(W)의 (i,j) 예시

-1, -1	0, -1	1, -1
-1, 0	0, 0	1, 0
-1, 1	0, 1	1, 1

$$NCC = \frac{\text{Cov}(W_L, W_R)}{\sqrt{\text{Var}(W_L) \cdot \text{Var}(W_R)}}$$

두 블록 사이의 공분산

$$\text{Cov}(W_L, W_R) = \frac{1}{N} \sum_{(i,j) \in W} (I_L(x+i, y+j) - \bar{I}_L)(I_R(x+i-d, y+j) - \bar{I}_R)$$

각 블록의 표준편차의 곱

$$\sqrt{\text{Var}(W_L) \cdot \text{Var}(W_R)} = \sqrt{\frac{1}{N} \sum (I_L - \bar{I}_L)^2} \cdot \sqrt{\frac{1}{N} \sum (I_R - \bar{I}_R)^2}$$

Disparity Estimation

- Block matching(블록 매칭) 기반 방법
 - 픽셀 주변의 작은 윈도우(Block) 단위로 유사도를 비교하는 방식
 - 블록 매칭의 장점
 - 알고리즘이 직관적이고 구현이 비교적 쉬움. 즉, 왼쪽 영상의 블록을 오른쪽 영상의 같은 행 (Scanline)을 따라 이동시키며 유사도(SAD, SSD 등)를 계산하기만 하면 됨
 - 연산 구조가 병렬 처리에 매우 유리함. GPU나 FPGA에서 수천 개의 블록을 동시에 계산할 수 있어 실시간(Real-time) 처리가 가능
 - 특정 탐색 범위 내의 데이터만 로드하면 되므로 메모리 점유율이 비교적 낮음

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



텍스처 부족 영역 (Textureless Region)

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



반복 패턴 (Repetitive Patterns)

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



가려짐 영역 (Occlusion region)

Disparity Estimation

- Block matching(블록 매칭) 기반 방법

- 블록 매칭의 단점

- 텍스처 부족 영역 (Textureless Region) 취약: 단색 벽면이나 하늘처럼 특징이 없는 곳에서는 모든 블록이 비슷함.
 - 반복 패턴 (Repetitive Patterns) 취약: 체커보드나 창문 격자처럼 같은 모양이 반복되는 곳에서는 어느 영역이 진짜 쌍인지 구분이 어려움
 - 가려짐 (Occlusion) 문제 취약: 한쪽 카메라에는 보이지만 다른 쪽 카메라에는 가려져서 보이지 않는 영역(Occlusion region)에 대해서는 블록 매칭으로 disparity 추정이 어려움



가려짐 영역 (Occlusion region)

Blocking Matching 실습

```
import cv2
import numpy as np
import time
import matplotlib.pyplot as plt
import urllib.request

# Aloe 이미지 다운로드
url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg'
urllib.request.urlretrieve(url1, 'aloeL.jpg')
urllib.request.urlretrieve(url2, 'aloeR.jpg')

imgL_color = cv2.imread('aloeL.jpg')
imgL_gray = cv2.imread('aloeL.jpg', cv2.IMREAD_GRAYSCALE)
imgR_gray = cv2.imread('aloeR.jpg', cv2.IMREAD_GRAYSCALE)

# 원본 해상도 저장
orig_h, orig_w = imgL_gray.shape

# StereoBM 설정 (SAD 기반)
num_disp = 128
block_size = 15

stereo = cv2.StereoBM_create(numDisparities=num_disp, blockSize=block_size)

# disparity 계산
start_time = time.time()
disparity = stereo.compute(imgL_gray, imgR_gray)
elapsed_time = time.time() - start_time

# StereoBM 결과는 16배 스케일이므로 실제 픽셀 단위로 변환
disparity_float = disparity.astype(np.float32) / 16.0

# 시각화를 위한 정규화 (0~255)
# 시차 값이 0보다 작은 영역(무효값)은 제외
disparity_vis = cv2.normalize(disparity, None, 0, 255, cv2.NORM_MINMAX,
cv2.CV_8U)

# 결과 출력
print(f"이미지 크기: {orig_w}x{orig_h}")
print(f"소요 시간: {elapsed_time:.4f} 초")

plt.figure(figsize=(20, 10))
```

Blocking Matching 실습

```
# 왼쪽 원본 이미지
plt.subplot(1, 2, 1)
plt.title("Original Left Image")
plt.imshow(cv2.cvtColor(imgL_color, cv2.COLOR_BGR2RGB))
plt.axis('off')

# 원본 크기의 Disparity Map
plt.subplot(1, 2, 2)
plt.title(f"Full-Res SAD Disparity Map\n(NumDisp: {num_disp}, Block: {block_size})")
plt.imshow(disparity_vis, cmap='jet')
plt.colorbar(fraction=0.046, pad=0.04)
plt.axis('off')

plt.tight_layout()
plt.show()
```

Disparity Estimation

- 2.5D 영상
 - 2D 영상 + 각 픽셀의 깊이(Depth) 정보가 결합된 형태의 영상
- 2D, 3D 영상과의 비교
 - 2D 영상: 각 픽셀이 색상 정보(R, G, B)만 가짐
 - 3D 영상: 사물의 모든 면(앞, 뒤, 옆)에 대한 좌표(X, Y, Z) 및 색상 정보가 있어 자유로운 회전이 가능
 - 2.5D 영상 : 카메라가 바라보는 시점에서의 앞면 정보만 존재하되, 각 픽셀이 카메라로부터 얼마나 떨어져 있는지(깊이 값)를 알고 있는 상태



2.5D 영상의 개념

OpenGL 실습

- .ply
- 스탠퍼드 대학의 그래픽스 연구실에서 3차원 스캐너의 데이터를 효율적으로 저장하기 위해 개발한 포맷
- 각 정점(Vertex)에 더 많은 속성(Property)을 부여하는 데 특화되어 있음
- 예시)

```
ply
format ascii 1.0
element vertex 3
property float x
property float y
property float z
property uchar red
property uchar green
property uchar blue
element face 1
property list uchar int vertex_indices
end_header
0.0 1.0 0.0 255 0 0 # 1번 정점: 좌표(0,1,0), 색상(빨강)
-1.0 -1.0 0.0 0 255 0 # 2번 정점: 좌표(-1,-1,0), 색상(초록)
1.0 -1.0 0.0 0 0 255 # 3번 정점: 좌표(1,-1,0), 색상(파랑)
3 0 1 2 # 면: 3개의 정점을 사용하며, 인덱스는 0, 1, 2
```

2.5D 영상 실습

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 1. 이전 단계에서 생성한 데이터 준비 (imgL_color, disparity_float 사용)
scale = 1.0
img_small = cv2.resize(imgL_color, None, fx=scale, fy=scale)
disp_small = cv2.resize(disparity_float, None, fx=scale, fy=scale)

h, w = disp_small.shape

# 초점거리(f)와 베이스라인(B)은 임의의 값을 설정
f = 0.8 * w # 가상의 초점 거리
B = 1.0     # 가상의 카메라 간격

# 3. 3D 좌표 계산
x_coords, y_coords = np.meshgrid(np.arange(w), np.arange(h))

mask = disp_small > 0.5
```

```
# Z = (f * B) / disparity
z = np.zeros_like(disp_small)
z[mask] = (f * B) / disp_small[mask]

# X, Y 좌표 역투영
x = (x_coords - w/2) * z / f
y = (y_coords - h/2) * z / f

# 4. 시각화 데이터 준비 (유효한 점만 필터링)
points_x = x[mask].ravel()
points_y = y[mask].ravel()
points_z = z[mask].ravel()

# 색상 정보 추출 (BGR -> RGB 및 0~1 정규화)
colors = cv2.cvtColor(img_small, cv2.COLOR_BGR2RGB)[mask].reshape(-1, 3) / 255.0
```

2.5D 영상 실습

```
def save_point_cloud_ply(filename, points_x, points_y, points_z, colors):  
    """  
    포인트 클라우드 데이터를 PLY 파일로 저장  
    points_x, y, z: 1차원 넘파이 배열 (좌표)  
    colors: 1차원 넘파이 배열 (0~1 범위의 RGB 값)  
    """  
    # 색상을 0~255 범위의 정수로 변환  
    colors_int = (colors * 255).astype(np.uint8)  
  
    # 헤더 작성  
    num_points = len(points_x)  
    header = f"""ply  
format ascii 1.0  
element vertex {num_points}  
property float x  
property float y  
property float z  
property uchar red  
property uchar green  
property uchar blue  
end_header  
"""
```

```
# 데이터 결합 및 저장  
with open(filename, 'w') as f:  
    f.write(header)  
    for i in range(num_points):  
        # 좌표와 색상을 한 줄씩 기록  
        f.write(f'{points_x[i]} {points_y[i]} {points_z[i]} {colors_int[i, 0]}  
{colors_int[i, 1]} {colors_int[i, 2]}\n')  
  
    print(f'성공: '{filename}' 파일이 저장되었습니다. (총 {num_points}개의 점)')  
  
# 파일 저장 실행  
save_point_cloud_ply("aloe_2_5d.ply", points_x, points_y, points_z, colors)
```

2.5D 영상 실습

- Meshlab
 - 메쉬(Triangular Meshes)와 Point Cloud와 같은 3D 모델 데이터를 편집, 정리, 복원 및 렌더링 하기 위해 개발된 오픈 소스 소프트웨어
 - Ply, obj 포맷 데이터 확인 때 많이 사용
 - 다운로드 : <https://www.meshlab.net/>



장점	단점
무료 및 오픈 소스: 누구나 제한 없이 사용 가능.	메뉴가 비교적 복잡
강력한 필터: 수백 개의 수학적 필터 제공.	대용량 데이터 처리 시 가끔 튕김.
광범위한 포맷: PLY, STL, OFF, OBJ, 3DS 등 거의 모든 포맷 지원.	직접 모델링은 불가

2.5D 영상 실습

- 2.5D 영상(컬러영상 + disparity map) 예시



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{data}(d)$ (Data Term) : 왼쪽 영상의 픽셀과 오른쪽 영상의 대응 픽셀이 얼마나 유사한지를 측정(유사도)
- $E_{smooth}(d)$ (Smoothness Term) : 인접한 픽셀끼리는 d 값이 비슷해야 한다는 제약 조건을 부여함.

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d) \quad d : \text{깊이지도에서의 disparity(양안시차)들}$$

- $E_{data}(d)$ (Data Term) : 왼쪽 영상의 픽셀과 오른쪽 영상의 대응 픽셀이 얼마나 유사한지를 측정(유사도)
- 즉, 왼쪽 영상의 픽셀과 오른쪽 영상의 대응 픽셀 간 유사도가 가장 클 때 $E_{data}(d)$ 는 최소가 됨
- 일반적으로 블록 매칭 결과와 같음
- 예) $E_{data}(d) = \sum_{(x,y)} SAD(x, y, d)$
- $E_{data}(d)$ (Data Term) 만 있다면 블록 매칭과 동일한 문제 발생

Disparity Estimation

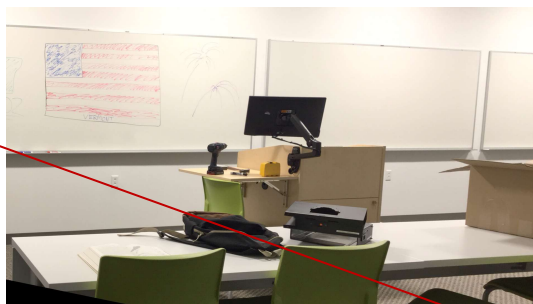
- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{smooth}(d)$ (Smoothness Term) : 인접한 픽셀끼리는 d 값이 비슷해야 한다는 제약 조건을 부여함.
- 노이즈를 억제하고 매끄러운 깊이지도를 만들



텍스처 부족 영역
(Textureless Region)



매끄러운 깊이값을 가짐



반복 패턴
(Repetitive Patterns)



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 깊이지도(disparity map, depth map)의 특징
 - 깊이가 변하는 지역을 제외하고 주변 픽셀의 d 값이 서로 비슷함(비슷한 깊이를 가지기 때문)

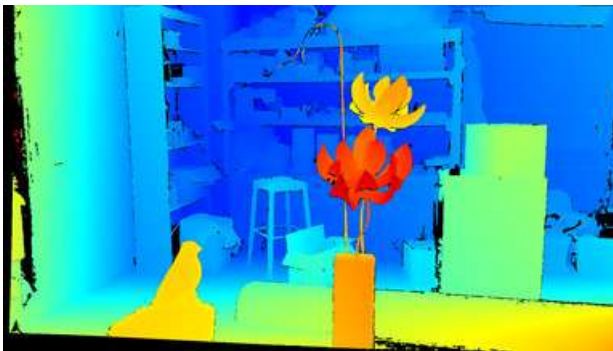
Left image



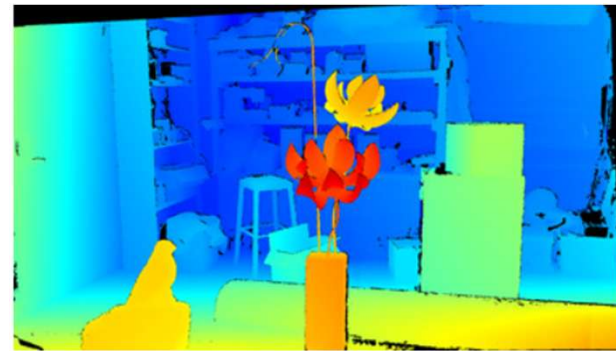
Right image



Disparity map of
left image



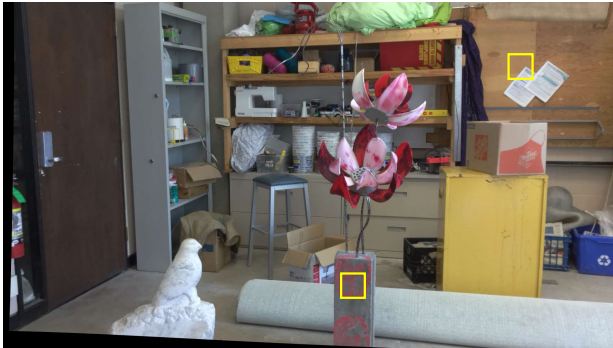
Disparity map of
right image



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 깊이지도(disparity map, depth map)의 특징
 - 깊이가 변하는 지역을 제외하고 주변 픽셀의 d 값이 서로 비슷함(비슷한 깊이를 가지기 때문)

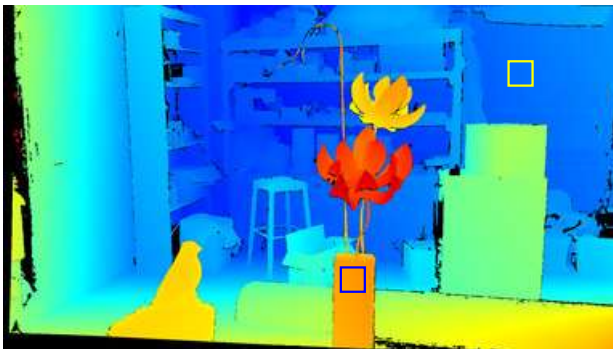
Left image



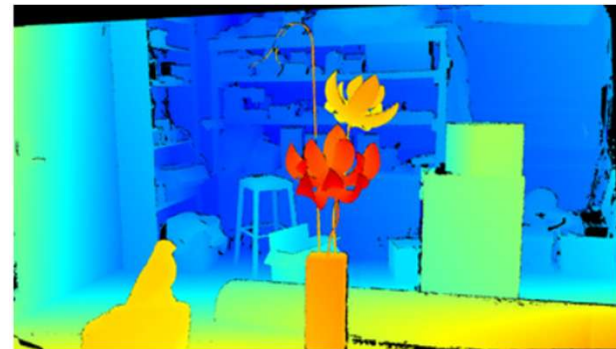
Right image



Disparity map of
left image



Disparity map of
right image



Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{smooth}(d)$ (Smoothness Term) : 인접한 픽셀끼리는 d 값이 비슷해야 한다는 제약 조건을 부여함.
- 즉, 인접한 픽셀 간 d 값의 차가 작을수록 $E_{smooth}(d)$ (Smoothness Term)는 작아짐
- 만약 깊이지도의 모든 d 가 동일하면 $E_{smooth}(d) = 0$ (최소) 가 됨
- 예) 인접한 두 픽셀 p, q 의 disparity를 d_p, d_q 라고 하면

1) $E_{smooth}(d_p, d_q) = \lambda \cdot [d_p \neq d_q]$: p, q 의 disparity가 다르면 무조건 λ 만큼 에너지가 증가

2) $E_{smooth}(d_p, d_q) = \lambda \cdot |d_p - d_q|$: p, q 의 disparity의 차가 클수록 에너지가 더 많이 증가

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - 영상의 깊이값들의 전체 에너지 $E(d)$ 를 최소화하는 방식으로 깊이지도 생성
 - 깊이지도를 찾는 문제를 최적화 문제로 모델링
 - 전체 에너지 $E(d)$ 는 아래와 같이 모델링 됨

$$E(d) = E_{data}(d) + E_{smooth}(d)$$

- $E_{data}(d)$ 와 $E_{smooth}(d)$ 간에는 trade-off 관계가 있음
- $E(d)$ 를 최소화하는 것은 $E_{data}(d)$ 와 $E_{smooth}(d)$ 간의 최적의 균형을 찾는 것
- $E(d)$ 를 최소화하는 최적의 깊이지도 d_{opt} 를 찾는 것은 현실적으로 불가능하기 때문에 다양한 최적화 방법으로 d_{opt} 의 근사해를 구함

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - GraphCut
 - 깊이 지도를 픽셀 간 그래프로 모델링하고 2차원 최적화를 수행
 - Dynamic programming
 - 각 라인 별로 매칭 비용에 대한 테이블을 만들고 DP로 최적해를 구함(1차원 최적화)



Stereo Image
(Left image)



Ground truth (정답)
disparity map



Block matching
(SAD)



Global matching
(GraphCut)



Global matching
(Dynamic Programming)

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - Dynamic programming
 - 각 라인 별로 매칭 비용에 대한 테이블을 만들고 DP로 최적해를 구함(1차원 최적화)
 - 예) 영상의 크기가 10x8이고 search range가 5라고 하면

[illegible]

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - Dynamic programming
 - 각 라인 별로 매칭 비용에 대한 테이블을 만들고 DP로 최적해를 구함(1차원 최적화)
 - 예) 영상의 크기가 10x8이고 search range가 5라고 하면

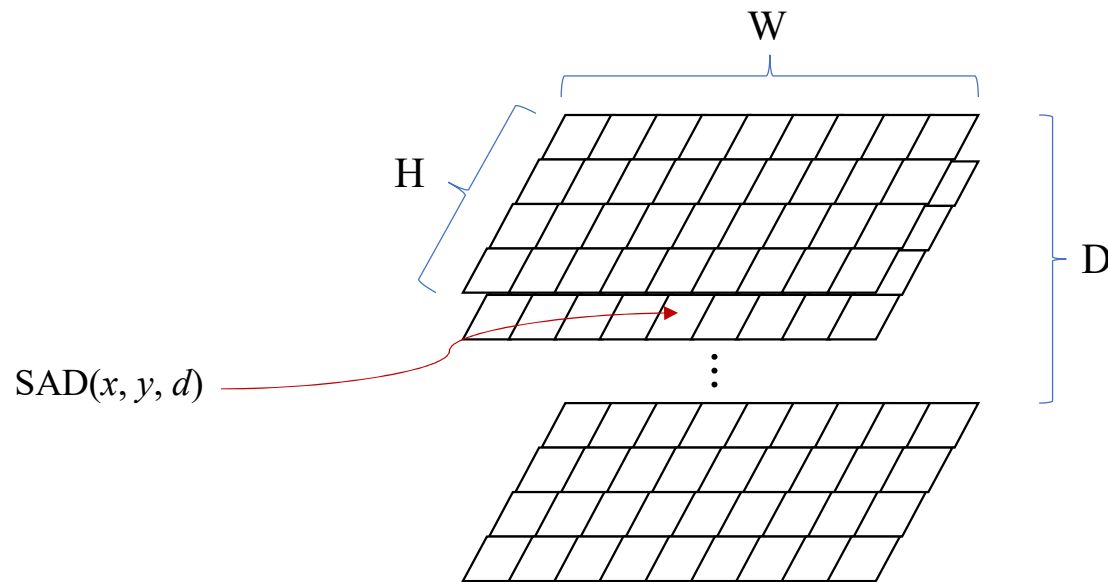
Search range {

SAD(0, 0)									
			SAD(x, d)						
									SAD(9, 4)

8개의 테이블에 대해 DP 수행

Disparity Estimation

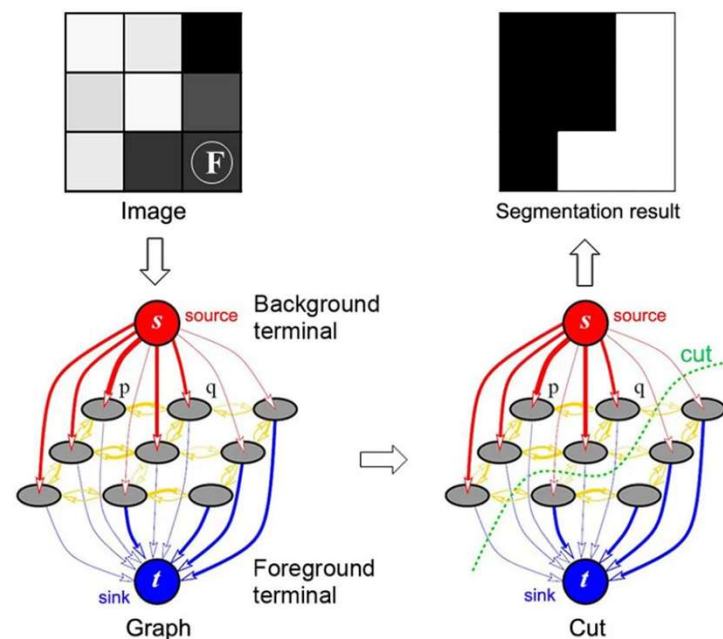
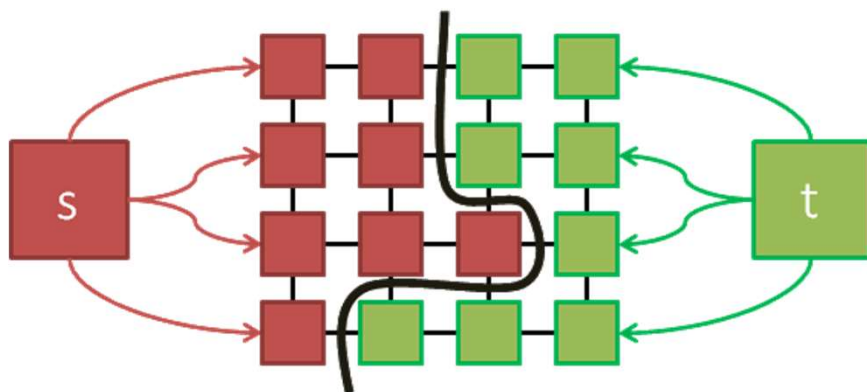
- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - GraphCut
 - 영상의 크기가 $W \times H$ 이고 search range가 D 라고 하면 $W \times H \times D$ 의 cost volume을 생성



cost volume의 예

Disparity Estimation

- Global matching(전역 매칭) 기반 방법
 - $E(d)$ 를 최소화하는 d 의 근사해를 구하는 대표적인 방법
 - GraphCut
 - 영상의 크기가 $W \times H$ 이고 search range가 D 라고 하면 $W \times H \times D$ 의 cost volume을 생성
 - Cost volume을 그래프로 모델링하고 minimum cut을 구함으로써 2차원 최적화를 수행



과제 #2

1. 과제#1 이후부터 현재까지 실습 코드를 모두 수행하고 결과 출력 (2점)
 - 각 수행마다 코드에는 어떤 과정인지 주석 처리

제출 방법 : LMS로 압축 파일 제출

- 코랩에서 다음과 같은 이름을 가진 노트북 파일을 생성한다.

ComputerVision_2ndHW_Class본인소속반_학번_영문이름.ipynb

예) ComputerVision_2ndHW_Class1_24xxxxx_HanshinLim.ipynb

- 생성된 코랩 노트북 파일(.ipynb)에서 코드를 구현 및 결과를 출력한다. 결과 출력시 어떤 결과인지를 명시한다.
- Due :
 - 1반 : 5월 11일 밤 12시 (이후 제출 0점)
 - 2반 : 5월 13일 밤 12시 (이후 제출 0점)

Disparity Estimation

- Semi-global matching(세미-전역 매칭) 기반 방법
 - 블록 매칭의 속도와 전역 매칭의 정확도 사이에서 균형을 맞춘 방식
 - Dynamic programming 기반 전역 매칭 방법은 한 라인에 대해서만 최적화를 수행하므로 라인 간 깊이 불일치로 인한 줄무늬가 생김
 - Semi-global matching에서는 8방향 또는 16방향에서 Dynamic programming을 수행하여 각각의 방향에 대해 경로 비용을 계산
 - 이후 모든 방향의 경로 비용을 더하고 이로부터 최적의 깊이 지도를 찾음

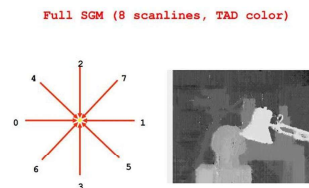
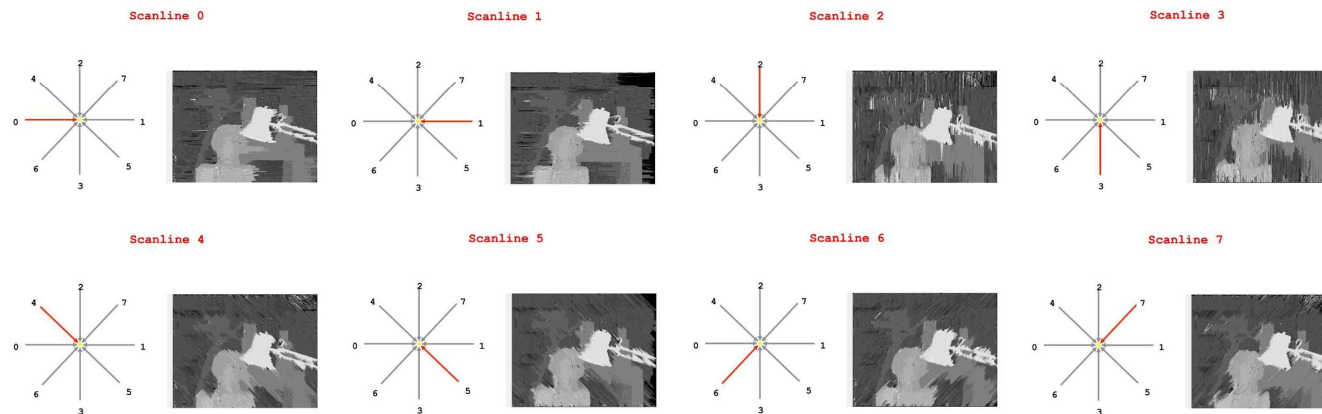
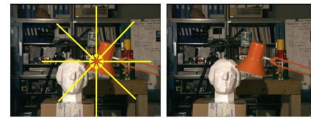
여러방향에서 수행

+ 비교적 고품질의 Disparity를 얻을 수 있다

+ 비교적 줄무늬가 덜 생김.

Disparity Estimation

- S



화를 수행하므로 라인 간 깊이 불
 lining을 수행하여 각각의 방향에
 음

Smoothness term의 예 : $C(p, d) = \begin{cases} 0 & \text{if } d_p = d_q \\ P_1 & \text{if } |d_p - d_q| = 1 \\ P_2 & \text{if } |d_p - d_q| > 1 \end{cases}$

$P_2 > P_1$

Disparity Estimation

- Semi-global matching(세미-전역 매칭) 기반 방법
 - 블록 매칭의 속도와 전역 매칭의 정확도 사이에서 균형을 맞춘 방식
 - 장점
 1. 여러 방향의 정보를 취합하므로 기존 Dynamic Programming의 문제인 가로 줄무늬가 거의 나타나지 않음
 2. 각 경로 계산이 독립적이어서 GPU나 FPGA를 통한 병렬화에 매우 유리

Semi-Global Matching 실습

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
import urllib.request
import time

url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg'
urllib.request.urlretrieve(url1, 'aloeL.jpg')
urllib.request.urlretrieve(url2, 'aloeR.jpg')
imgL = cv2.imread('aloeL.jpg')
imgR = cv2.imread('aloeR.jpg')

# 2. StereoSGBM 파라미터 설정
window_size = 3
min_disp = 0
num_disp = 16 * 8

# 모드 변경을 통한 속도 비교
# cv2.STEREO_SGBM_MODE_SGBM_3WAY (가장 빠름)
# cv2.STEREO_SGBM_MODE_SGBM (기본, 5방향)
# cv2.STEREO_SGBM_MODE_HH (8방향, 가장 느리지만 정밀)
```

```
current_mode = cv2.STEREO_SGBM_MODE_SGBM

stereo = cv2.StereoSGBM_create(
    minDisparity= min_disp,
    numDisparities=num_disp,
    blockSize=window_size,
    P1=8 * 3 * window_size**2,
    P2=32 * 3 * window_size**2,
    disp12MaxDiff=1,
    uniquenessRatio=10,
    speckleWindowSize=100,
    speckleRange=32,
    preFilterCap=63,
    mode=current_mode
)

# 3. 속도 측정 시작
start_time = time.time() # 시작 시각 기록

# 깊이 지도 계산
disparity_raw = stereo.compute(imgL, imgR)
```

Semi-Global Matching 실습

```
# 속도 측정 종료
end_time = time.time() # 종료 시각 기록
execution_time = (end_time - start_time) * 1000 # ms 단위로 변환

# disparity 정규화
disparity = disparity_raw.astype(np.float32) / 16.0

# 4. 결과 출력 및 시각화
print(f'SGBM Execution Time: {execution_time:.2f} ms')

plt.figure(figsize=(14, 7))
plt.subplot(121)
plt.imshow(cv2.cvtColor(imgL, cv2.COLOR_BGR2RGB))
plt.title('Left Image')
plt.axis('off')
plt.subplot(122)
plt.imshow(disparity, 'jet')
plt.title(f'Disparity Map (SGBM)\nTime: {execution_time:.2f} ms')
plt.axis('off')

plt.tight_layout()
plt.show()
```

□

Disparity Estimation

- Census Transform

- $n \times n$ 블록 내에서 기준 픽셀과 주변 픽셀과의 밝기를 비교하여 기준 픽셀보다 밝으면 1, 어두우면 0으로 변환
- 변환된 값들을 순서대로 나열하여 비트열 생성

예)

97	94	43
89	85	22
77	30	15

Census Transform

11000001

- 블록 매칭

- 기준 블록(왼쪽 영상)과 비교 블록(오른쪽 영상)에서 생성된 비트열끼리 hamming distance를 구함
- Hamming distance가 가장 작은 비교 블록을 매칭 블록으로 판정

예)

78	75	23
63	65	12
57	14	5

비교 블록

Census Transform

11000000

11000001
XOR 11000000

00000001

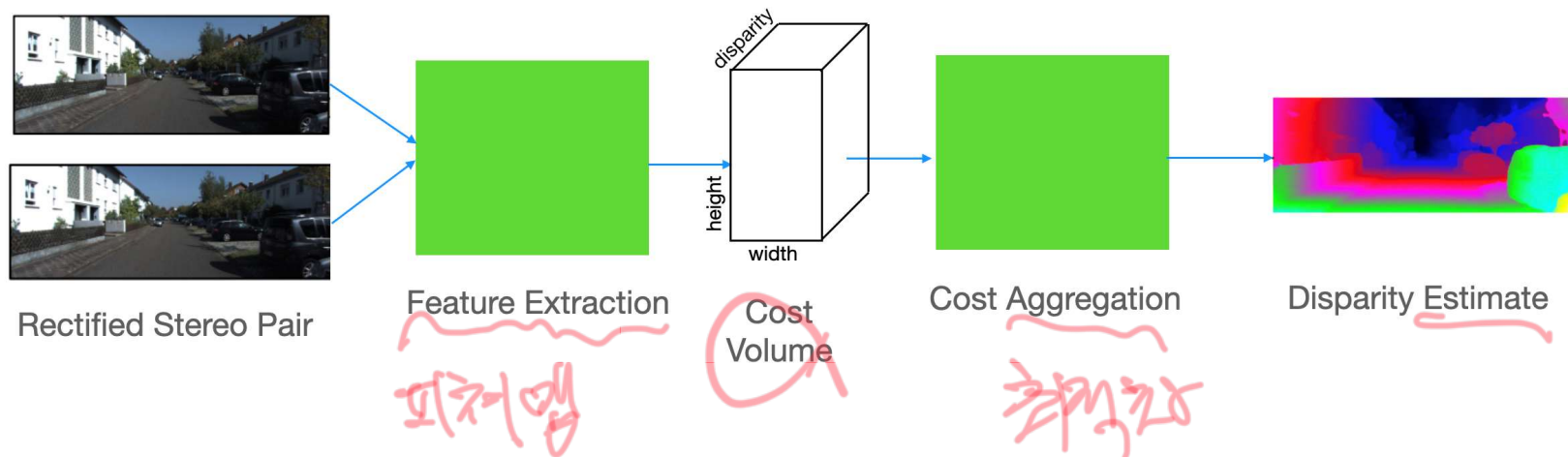
Hamming distance : 1

Disparity Estimation

- Census Transform
 - Census transform의 장점
 - 밝기 변화의 전체적인 패턴(Hamming Distance)은 크게 변하지 않아 결과가 안정적임
 - 주변보다 밝은지 어두운지의 순서(비트열)를 기록함으로써 매칭에 구조정보를 사용
 - 일반적인 연산보다 훨씬 적은 클럭 사이클로 매칭 비용을 계산할 수 있고 GPU 등의 병렬 처리에 적합

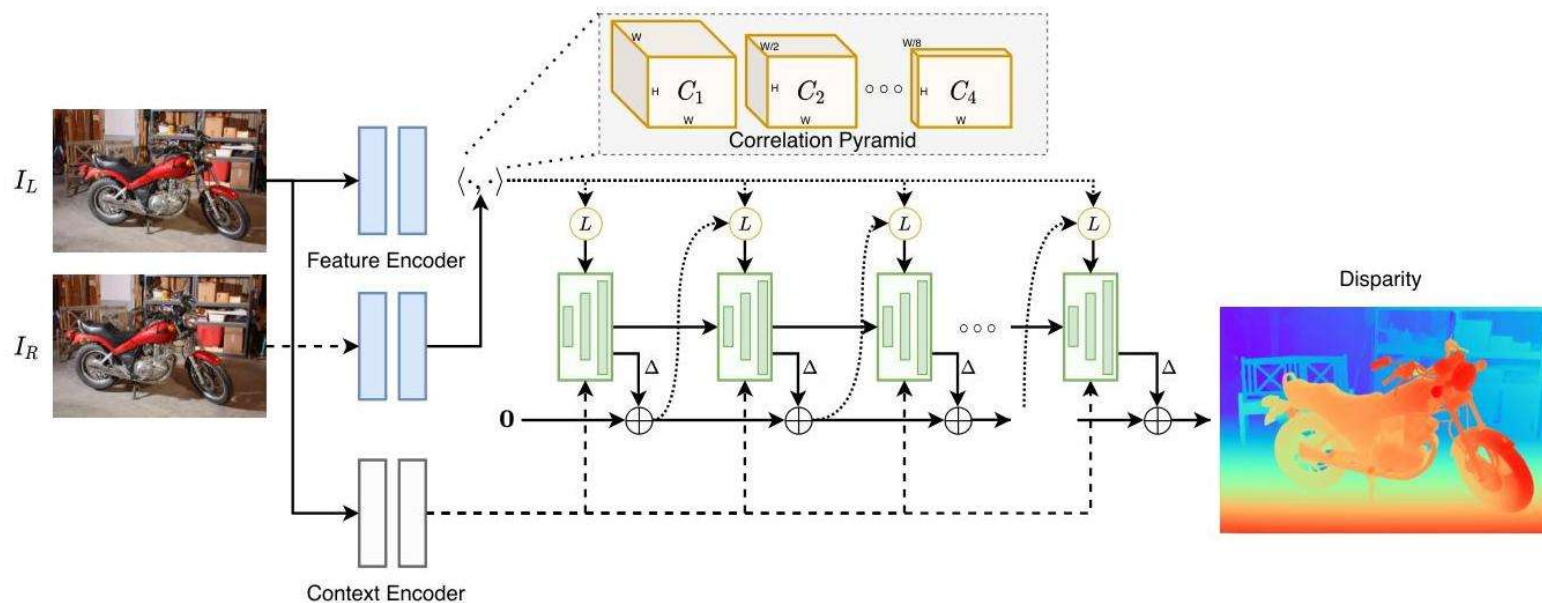
Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - 일반적으로 아래의 4단계로 구성됨
 1. 특징 추출(Feature extraction)
 2. Cost volume 생성 → 딥러닝의 생성
 3. 비용 최적화(비용 집계, cost aggregation)
 4. Disparity 추정



Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - Deep learning 기반 스테레오 매칭 알고리즘(RAFT-Stereo)의 예

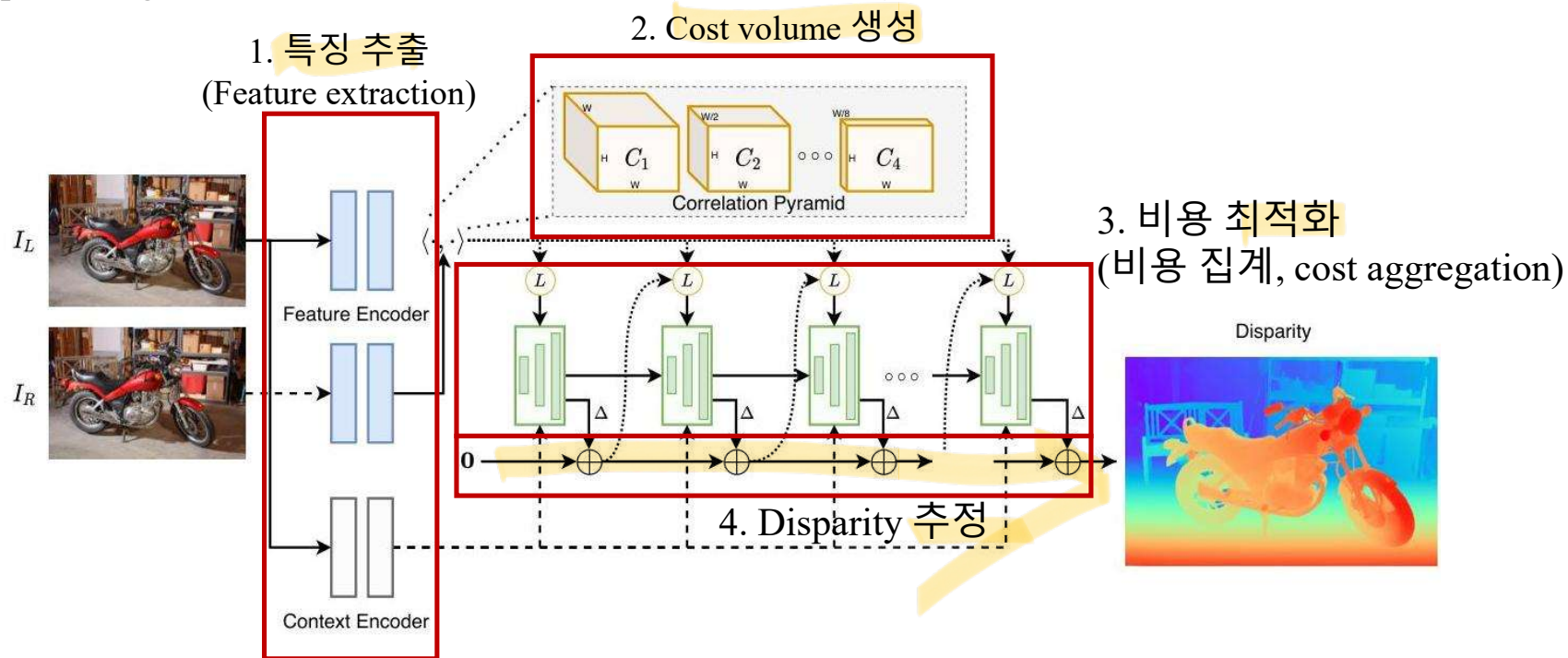


RAFT-Stereo 전체 구조

RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching, 3DV, 2021.

Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - Deep learning 기반 스테레오 매칭 알고리즘(RAFT-Stereo)의 예

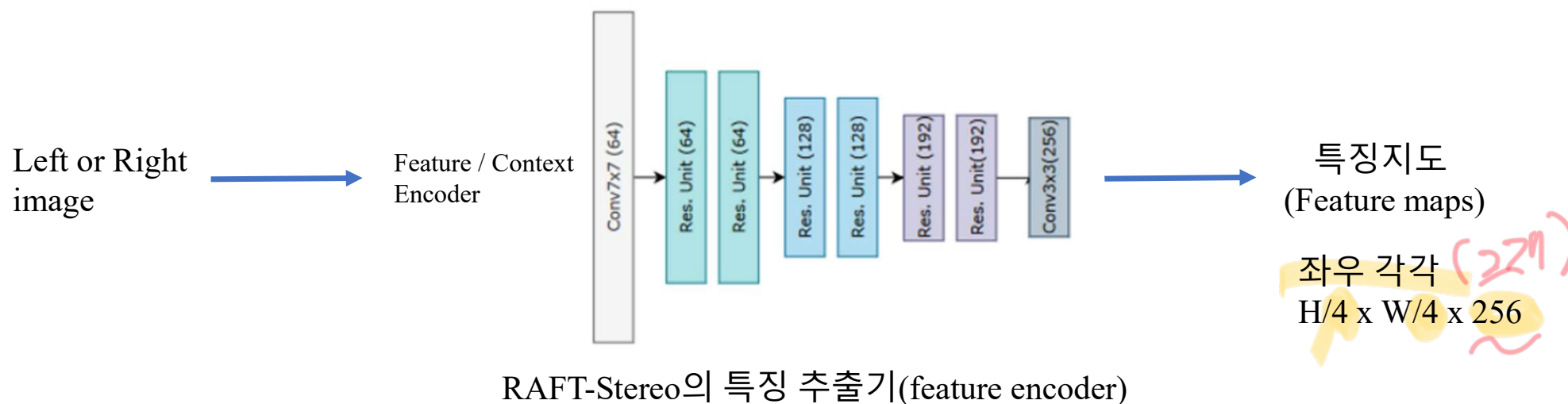


RAFT-Stereo 전체 구조

RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching, 3DV, 2021.

Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - 일반적으로 아래의 4단계로 구성됨
- 1. 특징 추출(Feature extraction)
 - 좌우 이미지 간의 유사도(Correlation)를 계산하기 위한 특징지도를 만듦
 - CNN Encoder 등을 사용하여 특징 추출
 - 예) RAFT-Stereo의 특징 추출기(feature encoder)



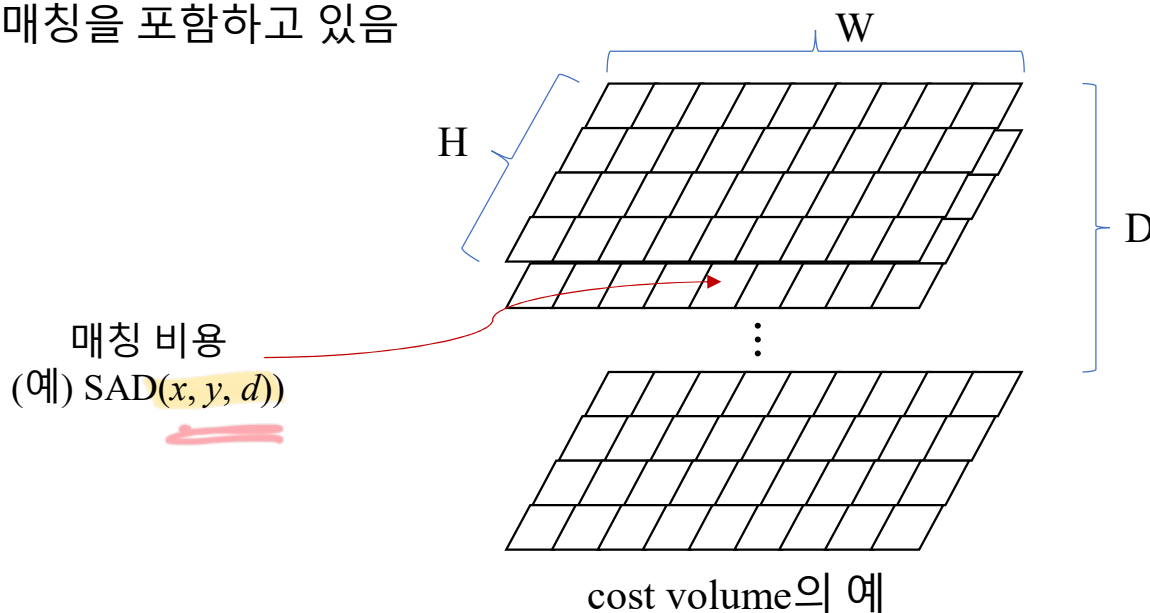
Disparity Estimation

- Deep learning(딥러닝) 기반 방법

- 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
- 일반적으로 아래의 4단계로 구성됨

2. Cost volume 생성

- Cost volume : 각 픽셀과 disparity에 대한 매칭 비용(matching cost)을 모아둔 데이터 구조
- 초기 cost volume의 매칭 비용은 픽셀 하나하나의 독립적인 정보만 담고 있어 노이즈가 많고 잘못된 매칭을 포함하고 있음



Disparity Estimation

- Deep learning(딥러닝) 기반 방법

- 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
- 일반적으로 아래의 4단계로 구성됨

2. Cost volume 생성(RAFT-Stereo의 예)

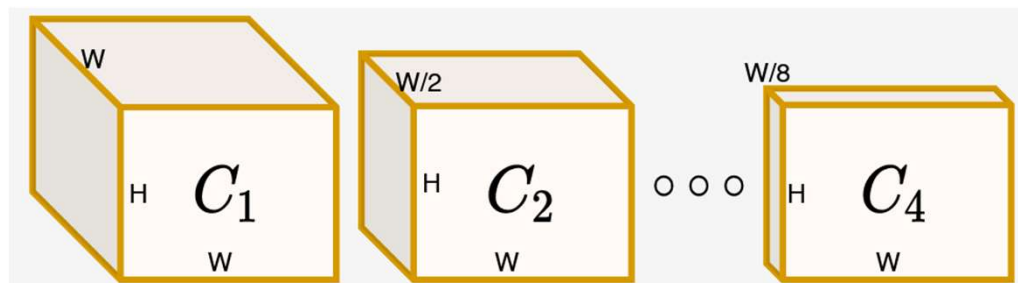
- 좌측 영상의 한 행(Row)에 있는 모든 특징 벡터와 우측 영상의 같은 행에 있는 모든 특징 벡터 간의 내적(Dot Product)을 구함. 즉,

$$C(x, y, d) = f_L(x, y) \cdot f_R(x - d, y) \quad f_L, f_R : \text{feature encoder 출력}$$

- 결과적으로 $W/4 \times H/4 \times W/4$ 크기의 텐서가 생성
- 마지막 차원 $W/4$ 에 대해 커널 크기 1, 2, 4, 8 등으로 Average Pooling을 수행하여 4개 층의 cost volume을 만들

(차단)
256개의 벡터를 내적

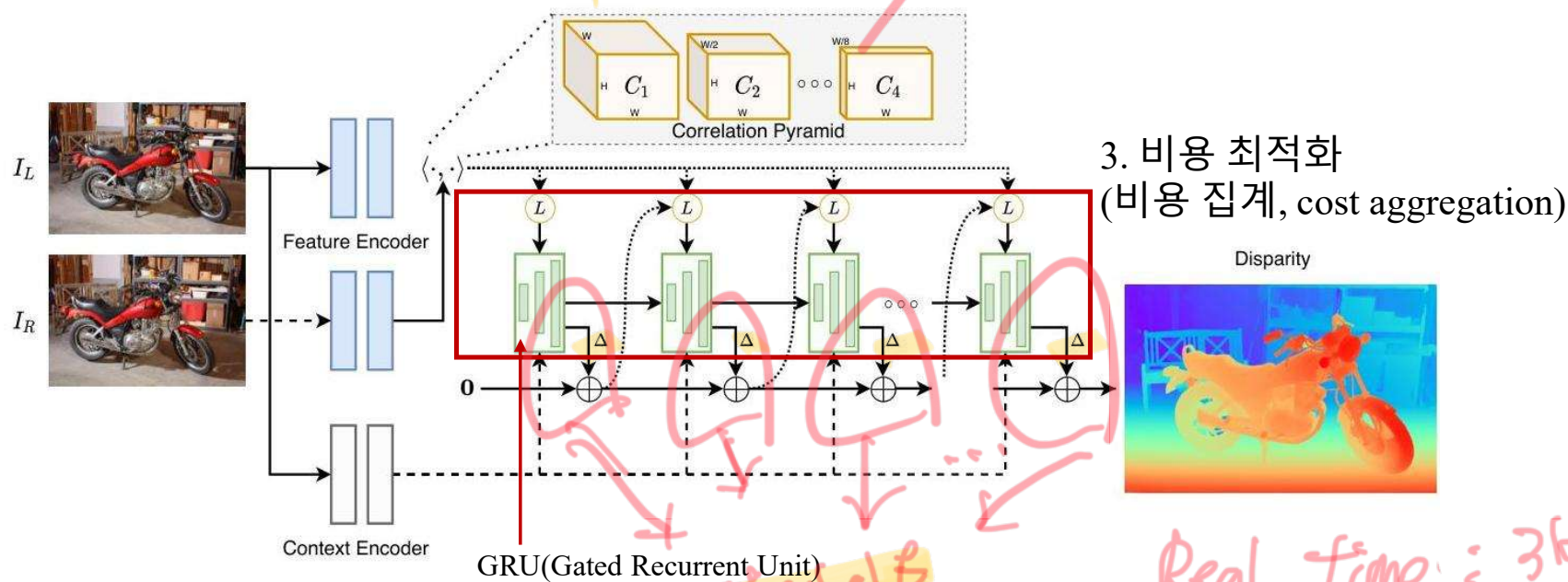
$$\begin{aligned} W &= w/4 \\ H &= h/4 \\ W &= w/4 \end{aligned}$$



RAFT-Stereo의 cost volume(correlation pyramid, 여기서 H, W는 실제로는 입력에서 /4)

Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - 일반적으로 아래의 4단계로 구성됨
- 3. 비용 최적화(비용 집계, cost aggregation)
 - 초기 cost volume을 3D CNN 또는 반복적 GRU 업데이트(RAFT-Stereo) 등으로 비용을 정제



4 만큼. 업데이트

Real time : 3번
최리밍동 : 30번이상

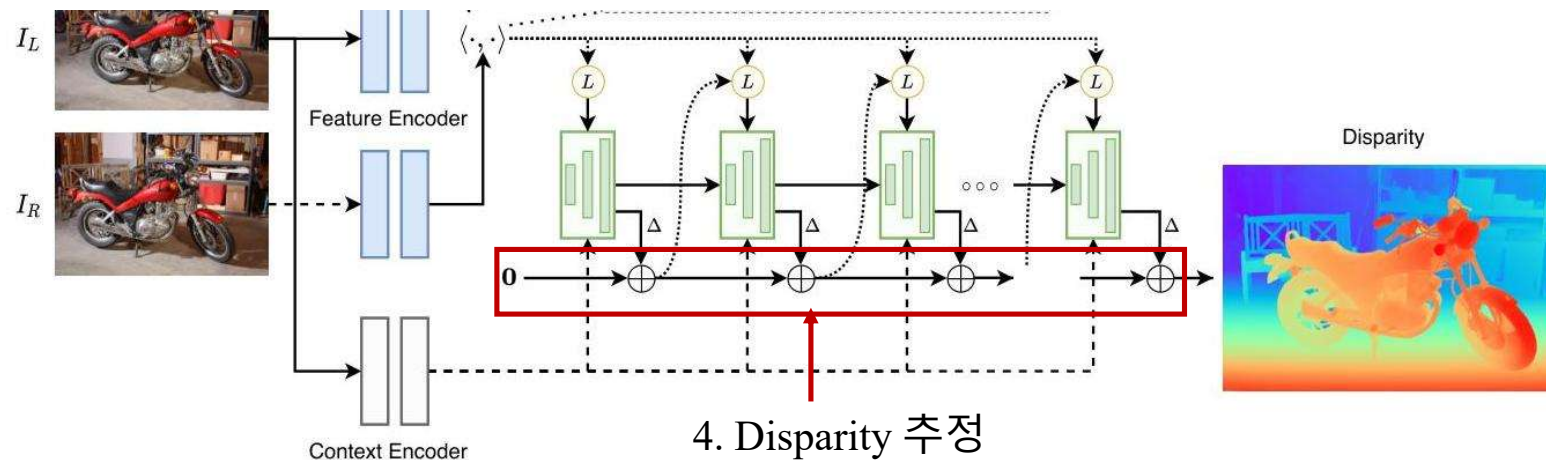
Disparity Estimation

- Deep learning(딥러닝) 기반 방법

- 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
- 일반적으로 아래의 4단계로 구성됨

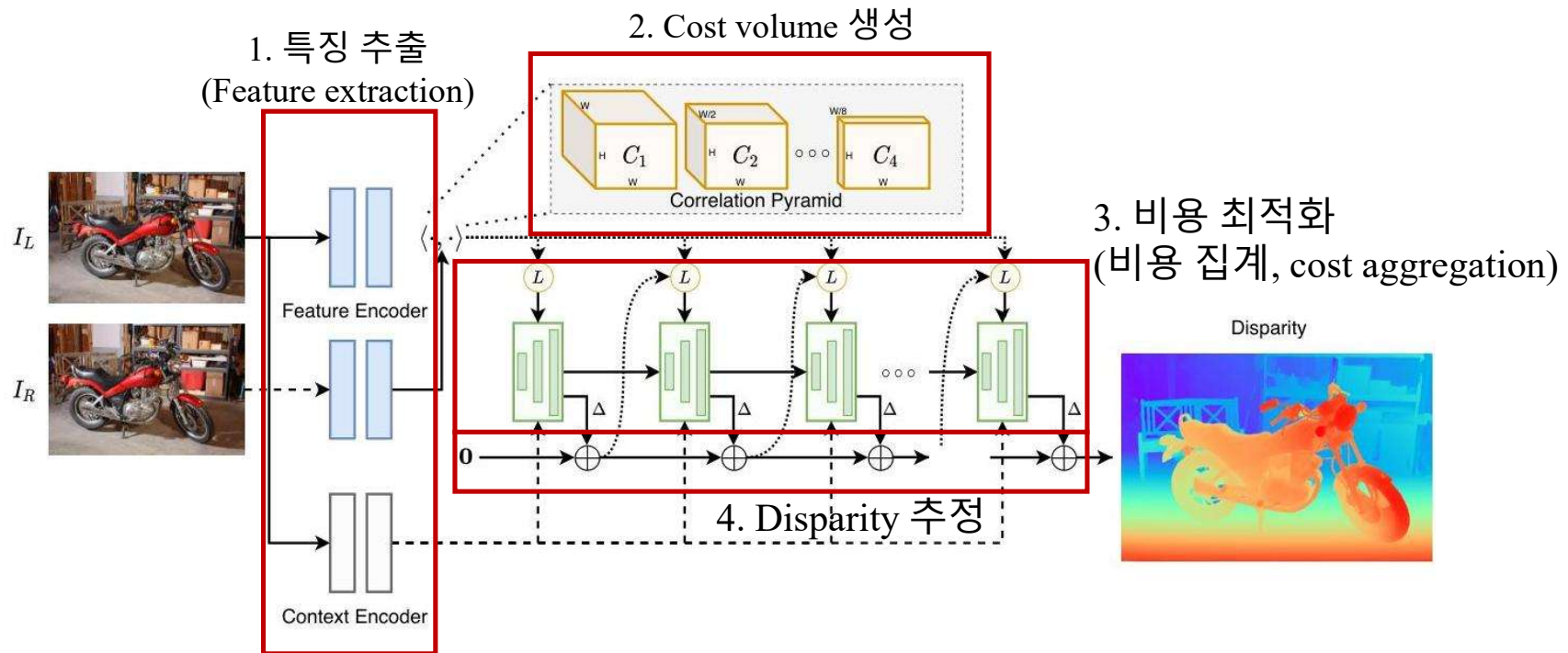
4. Disparity 추정

- Cost volume에서 정제된 비용(Cost)을 확률 분포로 변환한 뒤 모든 disparity의 후보들에 대해 확률만큼 가중평균하여 구함(기대값)
- RAFT-Stereo의 경우 GRU가 예측한 변화량(Δd)을 기존 disparity에 계속 더해서 최종값을 구함



Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - 두 영상으로부터 특징맵을 만들고 이로부터 영상의 깊이(양안시차)를 추정
 - Deep learning 기반 스테레오 매칭 알고리즘(RAFT-Stereo)의 예



RAFT-Stereo 전체 구조

RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching, 3DV, 2021.

Disparity Estimation

- Deep learning(딥러닝) 기반 방법
 - Deep learning 기반 방법의 장단점
 - 장점
 - 텍스처 부족, 텍스처 반복 영역 등에서 높은 정확도
 - 조명 변화에도 비교적 강인함
 - 단점
 - 일반화(generalization) 문제
 - 학습 데이터가 필요
 - 비교적 높은 컴퓨팅 파워(GPU, VRAM) 필요

데이터셋 부족, 체크포인트 부족
가중치/파라미터가 너무
(일반화X)

RAFT-Stereo 실습

[수정] -> [노트북 설정] -> [T4 GPU] 활성화

1. 환경 설정 및 저장소 클론

```
import os
```

```
# RAFT-Stereo 공식 repository 클론
```

```
!git clone https://github.com/princeton-vl/RAFT-Stereo.git
```

```
%cd RAFT-Stereo
```

```
# 필요한 라이브러리 설치
```

```
!pip install coremltools # 모델 구조에 따라 필요할 수 있음
```

2. Middlebury 데이터셋으로 사전 학습된 모델(Checkpoints) 다운로드

```
# 공식 모델 다운로드 스크립트 실행
```

```
!bash download_models.sh
```

```
# 다운로드가 완료되면 models/ 폴더 내에 체크포인트들이 생성됨
```

□

RAFT-Stereo 실습

```
import sys
sys.path.append('core')

import argparse
import time
import cv2
import numpy as np
import torch
from PIL import Image
from matplotlib import pyplot as plt
import urllib.request
from utils.utils import InputPadder
from raft_stereo import RAFTStereo

# 모델 로드 함수
def load_model(restore_ckpt):
    # 기본 설정 (기본값 사용)
    parser = argparse.ArgumentParser()
    args = parser.parse_known_args()[0]
    args.restore_ckpt = restore_ckpt
    args.mixed_precision = False
    args.slow_fast_gru = False
```

```
args.valid_iters = 32 # 반복 횟수 (정밀도 조절)
args.hidden_dims = [128]*3
args.context_dims = [128]*3
args.corr_implementation = "reg"
args.shared_backbone = False
args.corr_levels = 4
args.corr_radius = 4
args.n_downsample = 2
args.n_gru_layers = 3
args.context_norm = 'batch'
args.inter_num = 2

model = torch.nn.DataParallel(RAFTStereo(args))
model.load_state_dict(torch.load(args.restore_ckpt))
model = model.module
model.cuda()
model.eval()
return model
```

RAFT-Stereo 실습

4. 이미지 준비 및 추론

```
model = load_model('models/raftstereo-middlebury.pth')

url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg'
urllib.request.urlretrieve(url1, 'aloeL.jpg')
urllib.request.urlretrieve(url2, 'aloeR.jpg')
imgL = cv2.imread('aloeL.jpg')
imgR = cv2.imread('aloeR.jpg')

image1 = torch.from_numpy(imgL).permute(2, 0, 1).float()[None].cuda()
image2 = torch.from_numpy(imgR).permute(2, 0, 1).float()[None].cuda()

# 이미지 패딩 (모델 입력 크기 최적화)
padder = InputPadder(image1.shape)
image1, image2 = padder.pad(image1, image2)

with torch.no_grad():
    start_time = time.time()
    # 32회 반복 업데이트 수행
    _, flow_up = model(image1, image2, iters=32, test_mode=True)
    end_time = time.time()
```

결과 복원 (시차 값은 Flow의 음수값으로 표현됨)

```
disparity = -flow_up.cpu().numpy().squeeze()
execution_time = (end_time - start_time) * 1000
```

5. 시각화

```
from mpl_toolkits.axes_grid1 import make_axes_locatable

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

ax1.imshow(imgL)
ax1.set_title('Original Left Image')
ax1.axis('off')
div1 = make_axes_locatable(ax1)
div1.append_axes("right", size="5%", pad=0.1).axis('off')

im = ax2.imshow(disparity, cmap='jet')
ax2.set_title(f'RAFT-Stereo Disparity\nTime: {execution_time:.2f} ms')
ax2.axis('off')
div2 = make_axes_locatable(ax2)
cax = div2.append_axes("right", size="5%", pad=0.1)
plt.colorbar(im, cax=cax)
plt.show()
```